

**M A S A R Y K
U N I V E R S I T Y**

FACULTY OF INFORMATICS

**The tool implementing the “rare
k-mer” approach for alignment
filtering**

Bachelor's Thesis

PATRIK JUŘICA

Brno, Spring 2026

**MASARYK
UNIVERSITY**

FACULTY OF INFORMATICS

**The tool implementing the “rare
k-mer” approach for alignment
filtering**

Bachelor's Thesis

PATRIK JUŘICA

Advisor: Mgr. Monika Čechová, Ph.D.

Department of Machine Learning and Data Processing

Brno, Spring 2026



Declaration

Hereby I declare that this paper is my original authorial work, which I have worked out on my own. All sources, references, and literature used or excerpted during elaboration of this work are properly cited and listed in complete reference to the due source.

In accordance with the academic integrity guidelines of Masaryk University, I acknowledge the use of generative artificial intelligence (AI) tools during the preparation of this text. Specifically, an AI collaborator was utilised as a stylistical, editorial and programming assistant. The underlying research hypotheses, biological assumptions, algorithmic logic of the developed tool, and critical interpretation of the empirical results remain entirely my own authorial work.

Patrik Juřica

Advisor: Mgr. Monika řechová, Ph.D.

Acknowledgements

Many thanks belong to my advisor, Mgr. Monika Čechová, Ph.D., for her guidance and support during the process. I would also like thank my family and friends for their support.

Abstract

Although long-read sequencing technologies have significantly advanced the assembly and analysis of complex genomic structures, highly repetitive regions such as ribosomal DNA (rDNA) loci remain exceptionally challenging to resolve. Due to the nearly identical sequence homology across different rDNA arrays, standard read aligners frequently struggle with ambiguous placements. This limitation leads to a high rate of false-positive alignments, a problem further complicated by the elevated insertion and deletion (indel) error presence characteristic of long-read data. To address this computational bottleneck, this thesis explores and evaluates the "rare k-mer" approach as a secondary, post-alignment verification method. The core objective is the development of an efficient command-line filtering tool, implemented in Rust, that processes standard alignment data (BAM files). The algorithm verifies the true genomic origin of mapped long reads by assessing the positional compliance of sequence-specific rare k-mers. By benchmarking this filtering strategy against both simulated and real rDNA datasets, the thesis assesses the sensitivity, specificity, and computational efficiency of the approach. Ultimately, this work provides a functional open-source tool for confident read filtering and offers methodological guidance for improving the reliability of mutational analysis and read mapping in complex genomic regions.

Keywords

Bioinformatics, BAM, alignment filtering, rare k-mer

Contents

1	Introduction	1
	Introduction	1
1.1	Problem Statement	1
1.2	Goal of the Thesis	3
1.3	Structure of the Thesis	3
2	Biological and Technical Background	4
2.1	Ribosomal DNA (rDNA) Loci	4
2.1.1	Tandem repeats	4
2.2	Long-Read Sequencing Technologies	5
2.3	The Read Mapping Problem	7
2.4	K-mers in Bioinformatics	8
3	The Rare K-mer Approach	10
3.1	Concept and Rationale	10
3.2	Handling Sequencing Errors	11
3.3	Filtering Criteria	14
3.3.1	Defining Sequence Rarity	14
3.3.2	K-mer Length	15
3.3.3	Sequencing Error Handling	15
3.3.4	Verification Confidence	15
4	Implementation of the Filtering Tool	17
4.1	Choice of Technology	17
4.2	System Architecture and Workflow	19
4.2.1	Building the Rare K-mers Library	19
4.2.2	Alignment Filtering Workflow	19
4.2.3	The Evaluation Engine	20
4.2.4	Parallelisation	21
4.3	Algorithmic Details	21
4.3.1	Two-Bit DNA Encoding and Rolling Bitmask	21
4.3.2	Memory-Optimised Two-Pass Indexing	22
4.3.3	Tolerance Evaluation	22
4.4	Dynamic Tolerance	23
4.5	Command-Line Interface and Usage	24

4.5.1	I/O and Diagnostic Signatures	24
4.5.2	Dictionary Construction Boundaries	24
4.5.3	Evaluation and Evolutionary Scoring	25
4.5.4	Architectural Routing	25
5	Evaluation and Results	26
5.1	Methodology	26
5.1.1	Datasets	26
5.1.2	Hardware and Software	30
5.1.3	Evaluation Metrics	30
5.1.4	Finding Optimal Parameters	31
5.2	Filtering accuracy on Simulated Data	32
5.2.1	Perfect Reads	32
5.2.2	99% Reads	34
5.2.3	97% Reads	34
5.2.4	NanoSim Reads	39
5.3	Qualitative Evaluation on Sequencing Data	42
5.3.1	Experimental Setup and Parameter Selection . .	45
5.3.2	Data Reduction	45
5.3.3	IGV	46
5.4	Computational Efficiency	46
5.5	Discussion	49
5.5.1	Algorithm Efficacy and Robustness to Noise . .	49
5.5.2	Parameter Influence and Interaction	50
5.5.3	Biological recommendations	51
6	Conclusion	53
7	Appendices	54
	Bibliography	55

List of Tables

5.1	Reference arrays created at FI MU.	27
5.2	Overview of all Datasets	29
5.3	Parameters used in the conducted grid search.	33
5.4	Statistics of perfect read datasets pre-filtering.	33
5.5	Best results from a grid search on perfect data (sorted by highest F1).	34
5.6	Statistics of 99% reads datasets pre-filtering.	35
5.7	Best results from a grid search on 99% data (sorted by highest F1).	35
5.8	Best scoring parameters on 99% Maternal dataset (sorted by highest F1).	35
5.9	Best scoring parameters on 99% Paternal dataset (sorted by highest F1).	36
5.10	Statistics of 97% reads datasets pre-filtering.	38
5.11	Best results from a grid search on 97% data (sorted by highest F1).	39
5.12	Best scoring parameters on 97% Maternal dataset (sorted by highest F1).	39
5.13	Best scoring parameters on 97% Paternal dataset (sorted by highest F1).	39
5.14	Statistics of NanoSim reads datasets pre-filtering.	42
5.15	Best results from a grid search on NanoSim data (sorted by highest F1).	42
5.16	Best scoring parameters on NanoSim Maternal dataset (sorted by highest F1).	43
5.17	Best scoring parameters on NanoSim Paternal dataset (sorted by highest F1).	43
5.18	Parameter choices tested against sequencing data.	45
5.19	Resulting read counts after filtering.	46
5.20	Table showing relative computational cost for different combinations of k-mer length and distance tolerance.	49
5.21	Recommended "strict" parameter configuration.	52
5.22	Recommended parameter configuration for noisier datasets.	52

List of Figures

2.1	Location of rDNA on human chromosomes. [11]	5
3.1	A visualisation of tandem repeats, rare k-mers and aligned reads.	12
3.2	IGV view of a rDNA array and marked k-mers with frequency 5-10.	12
5.1	Heatmap showcasing relationship between percentage of intact k-mers and distance tolerance	36
5.2	Impact of distance and percentage on F1 shows inverse relationship, suggesting that room for error is necessary.	37
5.3	Impact of distance and percentage on F1 shows inverse relationship, suggesting that room for error is necessary (for completeness).	37
5.4	Different penalisation of substitution errors shows little to no effect on F1.	38
5.5	Heatmap showcasing relationship between percentage of intact k-mers and distance tolerance (97% dataset)	40
5.6	Impact of distance and percentage on F1 shows inverse relationship, suggesting that room for error is necessary (97% dataset).	40
5.7	Impact of distance and percentage on F1 shows inverse relationship, suggesting that room for error is necessary (for completeness, 97% dataset).	41
5.8	Different penalisation of substitution errors shows little to no effect on F1 (97% dataset). Substitution cost 2 shows only slightly higher prevalence of worse F1.	41
5.9	Heatmap showcasing relationship between percentage of intact k-mers and distance tolerance (NanoSim dataset). Noisier reads benefit from allowing lower percentage while lowering tolerance.	43
5.10	Impact of distance and percentage on F1. The differences between different distance values are less pronounced (NanoSim dataset).	44

5.11	Impact of distance and percentage on F1. The differences between different distance values are less pronounced (for completeness, NanoSim dataset).	44
5.12	Different penalisation of substitution errors and its effect on F1 (NanoSim dataset). Substitution cost 2 shows worse F1 scores on noisier data.	45
5.13	Screenshot of IGV showing retained reads after filtering with optimal configuration (Maternal array).	46
5.14	Screenshot of IGV showing retained reads after filtering with optimal configuration (Paternal array).	47
5.15	Heatmaps showcasing RAM consumption and CPU time based on parameter choice.	48
5.16	Relationship of distance tolerance to processing time (seconds) for different k-mer lengths.	48

1 Introduction

The complete assembly of the human genome represents one of the most significant achievements of the 21st century. Having a complete and reliable reference enables us to study genetic mutations with a high degree of confidence [1].

To leverage this reference, biological samples must first undergo DNA sequencing. During this laboratory process, the extracted genetic material is processed by a sequencing instrument, which produces large datasets of unplaced nucleotide sequences, commonly referred to as reads [2]. Because these reads — which vary in length depending on the sequencing technology — initially lack genomic context, a computational step known as read mapping (or alignment) is required. Alignment algorithms compare each sequence against the reference genome, assigning it to a specific genomic locus based on the highest sequence similarity [3]. The output of this mapping procedure is typically stored in standardised alignment formats, such as SAM or BAM, which record reads along with their precise mapped coordinates [4].

Thanks to concurrent advancements in sequencing technologies and mapping algorithms, the vast majority of the human genome is now computationally straightforward to align. The structural complexity and sequence uniqueness of these regions ensure that reads can be unambiguously assigned to their correct genomic loci. However, approximately 8% of the human genome consists of highly repetitive sequences that remain exceptionally challenging to map [5]. Notable among these regions is ribosomal DNA (rDNA), which serves as the primary focus of this thesis [1].

1.1 Problem Statement

While standard aligners perform exceptionally well across the majority of the genome, applying them to reads originating from rDNA frequently results in ambiguous placements and incorrect alignments. This is due to the nature of their alignment strategy: they deploy a “greedy” approach and manipulate the read in such a way that it distorts the details of the underlying sequence by forcing global align-

ments across near-identical yet distinct repeat units [6]. This distorts the true, fine-scale variations of individual rDNA copies by mistakenly representing them as artificial insertions, deletions (indels), or local misalignments.

Complicating the task even further, standard short-read sequencing is completely insufficient for the assembly and mapping of rDNA. It yields reads of length 150-500 base pairs (bp) [7], and if such a read falls somewhere in the tandem repetition (length of 43kb), it can be aligned to multiple hundreds of places within the rDNA array.

To study these regions, a next-generation sequencing technology yielding longer reads must be used, such as Oxford Nanopore, which generates reads of lengths in 10s of kbp (N50 > 100 kb, read lengths up to 882 kb) [8].

While the length of ultra-long reads is sufficient to bridge the gaps between tandem repetitions, the technology still exhibits higher error rates than standard short-read sequencing. Although modern chemistries have drastically reduced these rates to often below 2% [2, 8], this residual noise continues to increase the difficulty of accurate mapping by another factor.

Consequently, when a standard aligner processes a noisy long read originating from an rDNA array, it struggles to distinguish between genuine biological variations—which differentiate the homologous rDNA copies—and mere sequencing errors. This perfect storm of high sequence homology, greedy algorithmic heuristics, and elevated indel error rates culminates in a high frequency of false-positive alignments, where reads are confidently but incorrectly mapped to decoy loci (non-target genomic regions with sufficient sequence similarity to mistakenly attract read alignments). To ensure the reliability of mutational analysis and assembly in these complex regions, relying solely on primary alignment algorithms is insufficient. Recent advancements in structural validation, such as the GAVISUNK approach [9], have demonstrated the high efficacy of utilising unique *k*-mer anchors (SUNKs) combined with the spanning power of ultra-long reads to detect assembly collapses and misjoins. Similarly, for highly homologous tandem repeats, a robust, secondary computational verification method is critically required to validate the true genomic origin of these mapped long reads.

1.2 Goal of the Thesis

To address these challenges, this thesis explores a novel post-alignment verification method: filtering mapped long reads based on the presence of rare k-mers.

Even within highly homologous repetitive arrays, there exist occasional sequence-specific variations—unique or nearly unique k-mers—that can serve as anchors to verify a read’s true genomic origin.

We implement a command-line tool applying this strategy with possible modifications to the algorithmic functioning of the filtering. Then we present an evaluation and benchmarking of this tool against real and simulated long-reads on rDNA datasets. Finally, we provide recommendations for improving the reliability of future read mapping pipelines and mutational analyses in complex genomic loci, based on the performance of the rare k-mer approach.

1.3 Structure of the Thesis

Following this introduction, Chapter 2 establishes the biological and technical background required to fully contextualise the rare k-mer approach.

Building upon this foundation, Chapter 3 is concerned with specific details of the approach taken in this thesis.

In chapter 4, the implementation of the tool is described, with both internal algorithmic functioning of the tool and user-side control mechanisms for fine-tuning the strictness of the alignment verification.

Chapter 5 discusses the methodology used for verification and validation of the filtering strategy, provides the results obtained and guidance on the usage of this approach.

Finally, Chapter 6, Conclusion, summarises the core contributions and findings, outlining possible extensions.

2 Biological and Technical Background

This chapter renders the core biological and technical concepts we will discuss and build upon further in this work. First, we will briefly cover what rDNA is and its role within the genome. Having that basis, we will then explore the tandem repeats located in these regions. Next, we will give a general overview of the long-read sequencing technologies used for research in these areas and compare them to standard short-read sequencing methods. Building upon this comparison, we will then show how standard mappers struggle when used to map to these regions, even with long reads. Finally, we will establish a definition of k-mers that we will use in the rest of the thesis.

2.1 Ribosomal DNA (rDNA) Loci

Ribosomes are often described as the “protein factories” of human cells. Their main function is to synthesise proteins according to the genetic information encoded in the messenger RNA (mRNA) and transfer RNA (tRNA), which then generate building blocks for the whole organism [10].

They consist of ribosomal RNA (rRNA) and ribosomal proteins. rRNA is responsible for holding the mRNA and tRNA sequences in the correct positions, which prepares them for a process called translation and acts as a catalyst.

During translation, the ribosomal proteins supply the amino acids, which are then linked to form proteins encoded by the translated sequences.

In humans, the genes that encode rRNA are located on the short arms (p-arms) of the five acrocentric chromosomes: 13, 14, 15, 21 and 22, as can be seen in Figure 2.1 [11].

2.1.1 Tandem repeats

The regions encoding rRNA mainly consist of tandem repeats — structurally similar sequences that are identically or nearly identically copied from the beginning to the end and are placed directly next to each other. The length of the repeats in human rDNA is approximately

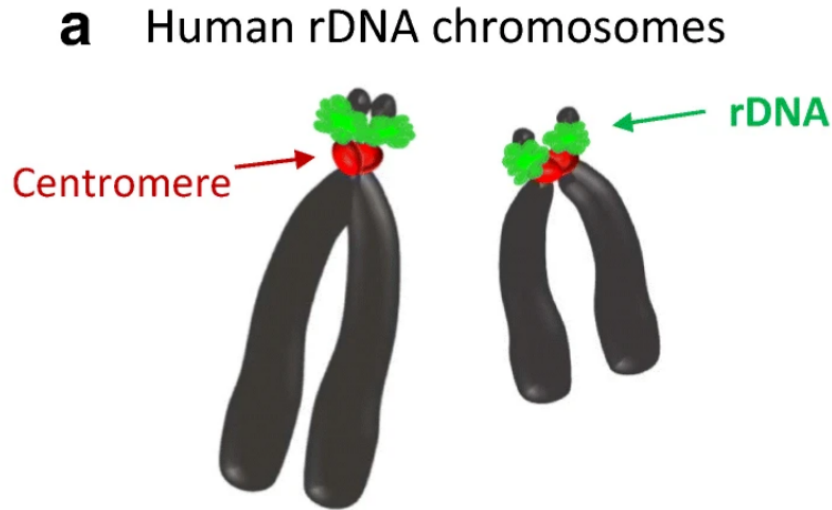


Figure 2.1: Location of rDNA on human chromosomes. [11]

43 kilobasepairs (kbp). A typical diploid human genome contains hundreds of copies of these units, arranged in tandem [12, 13].

Homologous sequences are regions of DNA that share a common origin, resulting in a high degree of structural and functional similarity. The observed homology of rDNA in humans is exceptionally high. The tandem repeats are nearly identical, not only within a single array (intra-chromosomal homology), but across the arrays of the five aforementioned chromosomes (inter-chromosomal homology) [14].

It is precisely due to this similarity across the genome that they represent one of the most difficult challenges for reference assembly and mutational analysis.

2.2 Long-Read Sequencing Technologies

The era of genomic analysis began in 1977 with the Sanger method, which produced reads of 500–900 bp. However, the discipline rapidly accelerated in the 2000s with the advent of Next-Generation Sequencing (NGS) technologies, particularly Illumina. Illumina sequencers established themselves as the industry standard due to their extremely

low error rates and significantly reduced costs. Their primary drawback, however, was a restricted read length of 150 to 500 bp [7, 15].

Due to this restricted read length, highly repetitive genomic loci — such as the long 43 kb tandem arrays of rDNA — remained largely unresolved. Short reads lack the necessary spatial context to span these extended repeats. Consequently, standard short-read assemblies inherently collapse, fragment, or produce highly ambiguous alignments at these complex loci. Overcoming this bottleneck required a shift to a novel sequencing technology, which produced continuous reads long enough to bridge these regions.

This need for extended read lengths drove the development of Third-Generation Sequencing (TGS) technologies, which were pioneered primarily by Pacific Biosciences (PacBio) and Oxford Nanopore Technologies (ONT). As the datasets analysed in this thesis are generated via ONT, the focus will remain on the characteristics of nanopore sequencing. Modern ONT platforms are capable of generating ultra-long continuous reads spanning tens to hundreds of kilobases (N50 values exceeding 100 kb and maximum lengths reaching up to megabase-pairs) [16], which therefore makes it sufficient to bridge our regions of interest.

Despite this advantage, long-read sequencing technologies inherently struggle with higher base-level error rates compared to their short-read counterparts. The current sequencing kits produce reads with an error rate of approximately 1%, or even less [17]. Nevertheless, this error margin remains roughly an order of magnitude higher than the 0.1% error rate characteristic of the best Illumina sequencing instruments [18].

The limitation of ONT data lies not only in the overall frequency of errors, but in the specific error profile. While Illumina platforms are prone to single-nucleotide substitution errors (SNPs) [19], nanopore sequencing suffers from insertion and deletion (indel) errors [20]. These artificial indels alter the length and the continuity of the read, which in turn complicates the mapping and assembly.

2.3 The Read Mapping Problem

To optimise computational efficiency against massive reference genomes, standard read aligners employ a heuristic approach known as the ‘seed-and-extend’ algorithm. This process is generally divided into three distinct phases [21]:

1. Seeding: The algorithm extracts short, fixed-length exact matches (seeds or k-mers) from both the reference genome and the sequencing reads. The positions of the reference seeds are stored in a highly optimised index data structure.
2. Chaining: The aligner queries the seeds of the reads against the reference index. It looks for co-linear clusters—groups of matching seeds that appear in the same relative order and distance in both the read and the reference. Identifying the most optimal chain establishes the read’s probable macroscopic alignment ‘neighbourhood’.
3. Extending: Once the optimal chain is selected, the algorithm performs a detailed, base-by-base sequence alignment to connect the seeded anchors. It calculates a final alignment score by mathematically accounting for local variations, such as insertions, deletions, and substitutions, thereby assigning the read to its final mapped coordinates.

While this heuristic approach is highly efficient for the vast majority of the reference genome, it fundamentally breaks down at the intersection of highly homologous loci and error-prone long reads. Because the rDNA arrays share near-perfect inter-chromosomal sequence identity, the seeding phase identifies equally viable anchor chains across tens of different genomic locations. Therefore, the algorithm is forced to rely on the scoring metric of the extension phase to determine the read’s true origin [22].

However, the high frequency of indels (inherent to nanopore sequencing) critically alters this calculation. The aligner cannot computationally distinguish between a true biological sequence variation and sequencing noise. Consequently, an artificial error in the read may yield a higher alignment score against a structurally similar non-target repeat, leading the algorithm to confidently assign the read to

an incorrect locus. This results in a high prevalence of false-positive alignments.

2.4 K-mers in Bioinformatics

To process and analyse massive biological sequences, bioinformatics relies heavily on a data structure known as the k-mer. A k-mer is defined as a contiguous substring of length k extracted from a larger biological sequence, such as DNA or RNA. The variable k denotes the number of nucleotides contained within that sequence fragment.

The extraction of k-mers from a sequence is achieved using a sliding window approach. A window of size k is placed at the beginning of the sequence to capture the first k-mer. The window is then shifted one base pair to the right to capture the next, and the process repeats until the end of the sequence is reached. Mathematically, for any given continuous sequence of length L , this sliding window process will systematically generate exactly $L - k + 1$ overlapping k-mers. Given the four-letter alphabet of DNA (A, C, G, T), the theoretical sequence space - the total number of mathematically possible unique combinations for any given k - is 4^k .

The primary use case for the k-mer-based approach is computational efficiency. Comparing two massive sequences base-by-base using traditional dynamic programming algorithms is highly accurate, but often computationally unfeasible for entire genomes.

Standard use cases include [23]:

- **Assembly:** Short-read assemblers rely heavily on k-mers to reconstruct genomes from scratch. By breaking reads into k-mers, the software constructs a graph data structure (De Bruijn graph, string graphs, etc.). The assembler navigates this graph by finding overlapping k-mers to stitch fragmented reads back into continuous DNA arrays.
- **Read Mapping:** As discussed in Section 2.3, modern aligners rely on a 'seed-and-extend' heuristic. The "seeds" extracted during the initial phase of alignment are simply k-mers. The aligner indexes the reference genome by its constituent k-mers to enable quick similarity search and subsequent mapping.

Apart from serving as the structural building block of algorithms, k-mers can also serve as a useful metric to assess genomic complexity. The number of distinct k-mers tells us about the nature of the given loci. If a k-mer occurs just once, it is likely a unique gene or a mutation, whereas if a k-mer sequence appears hundreds or thousands of times, it is indicative of a highly repetitive element.

In the context of a human genome, k-mers extracted from the aforementioned tandem arrays of rDNA are usually present in larger counts due to the extreme intra- and inter-chromosomal homology of these regions. The reliance of aligners on frequent k-mers and constructed neighbourhood forces them to choose between hundreds of equally viable loci. This results in the decoy problem previously described and creates the need for a novel approach to post-alignment verification.

3 The Rare K-mer Approach

As established in the previous chapter, standard alignment algorithms struggle to accurately map reads within highly homologous genomic regions like rDNA arrays. Because these aligners rely on highly frequent k-mers to establish a mapping neighbourhood, they are forced to choose between hundreds of equally viable loci, which generates false-positive alignments. To resolve this mapping ambiguity, this thesis proposes a post-alignment verification strategy that shifts the focus from sequence abundance to sequence rarity.

This chapter details the theoretical foundation of this rare k-mer approach. Firstly, we introduce the core rationale of the method, explaining how isolated, sequence-specific variations serve as biological anchors to verify the true origin of a read. Then we address the challenge of applying exact-match k-mer logic to error-prone Oxford Nanopore data by accounting for insertions and deletions. Finally, we define the theoretical basis for the specific filtering criteria and parameters used to evaluate and validate mapped reads.

3.1 Concept and Rationale

While the human reference genome has seen significant advancements, most notably the Telomere-to-Telomere (T2T) CHM13 assembly, the complexity of ribosomal DNA arrays presents an ongoing challenge. In current standard reference releases, these highly repetitive regions are heavily computationally modelled rather than continuously sequenced and assembled at a strict, base-by-base resolution across all copies [1]. Consequently, within the public T2T-CHM13 reference, the rDNA tandem repeats appear entirely identical, lacking any sequence-specific variation.

However, biological principles of incomplete concerted evolution, alongside preliminary localised assemblies — such as those of the chromosome 14 rDNA array created at FI MUNI [24] — suggest a different reality. It is strongly hypothesised that these repeat units are not perfectly identical *in vivo*. Rather, individual repeat units likely escape complete homogenization over evolutionary time, accumulating isolated, unit-specific mutations such as SNPs or small indels.

Distinguishing these true biological variations from standard long-read sequencing errors remains an open challenge, but their existence forms the foundational assumption of this work.

Operating under this biological assumption, if one analyses a true, unmodeled sequence of these rDNA copies using the k-mer extraction process described in Chapter 1, these isolated mutations would give rise to a distinct class of substrings: rare k-mers. While the vast majority of k-mers generated from an rDNA array would exhibit extremely high frequencies — mirroring the tandem repeat copy number — a sliding window passing over a unit-specific mutation generates a sequence fragment existing uniquely, or almost uniquely, at that specific locus. These hypothetical rare k-mers break the monotony of the highly homologous background.

The core rationale of the post-alignment filtering approach proposed in this thesis is predicated on the existence of these sequence-specific variations. The tool is designed to leverage these rare k-mers to verify read alignments. When a standard aligner maps an Oxford Nanopore read to a specific locus within the rDNA array, it makes a statistical assumption based on the overall sequence similarity score. The rare k-mer approach tests this assumption by querying the underlying reference sequence of the proposed mapping coordinate.

If the custom reference locus contains sequence-specific rare k-mers, a read that genuinely originates from this exact position must also contain those matching anchors. Conversely, if the read is a false-positive decoy — originating from a different rDNA repeat but incorrectly mapped to the current locus due to algorithmic heuristics — it will lack the specific rare k-mers dictated by the reference locus (see Figures 3.1, 3.2). By computationally scanning the mapped reads for the presence and precise correspondence of these anchors, the tool shifts the validation metric. Rather than relying on a global sequence similarity score, it aims to verify alignments using localised, high-confidence genomic signatures.

3.2 Handling Sequencing Errors

While the theoretical framework of utilising rare k-mers as biological anchors is conceptually robust, its practical application is immedi-

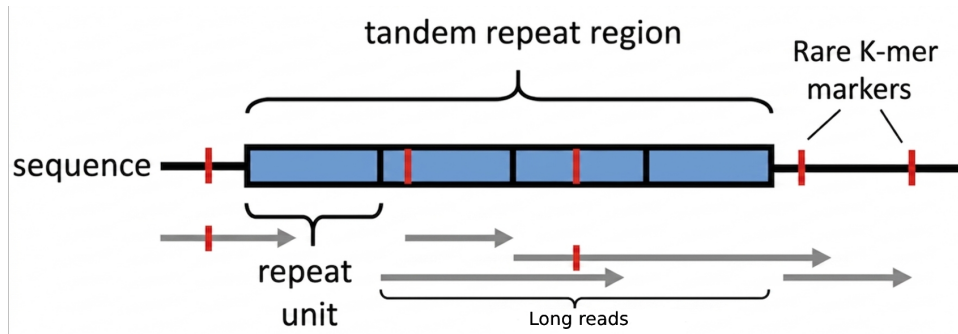


Figure 3.1: A visualisation of tandem repeats, rare k-mers and aligned reads.

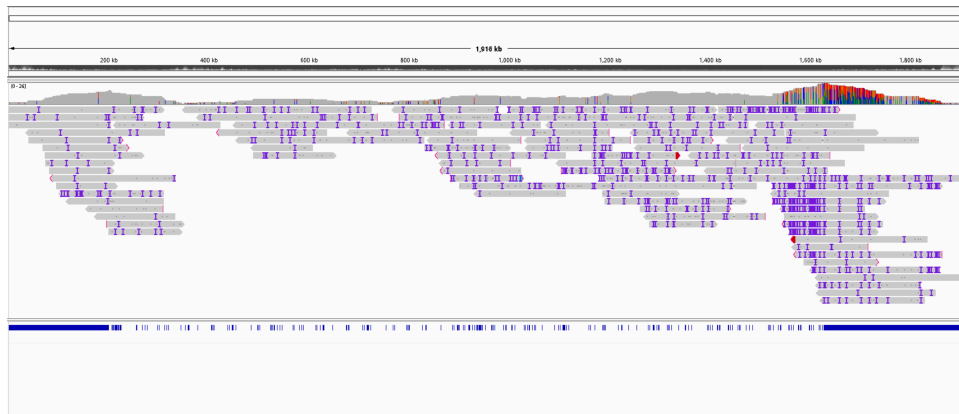


Figure 3.2: IGV view of a rDNA array and marked k-mers with frequency 5-10.

ately challenged by the physical limitations of current sequencing technologies. In ideal, error-free datasets, verifying the presence of a reference-derived k -mer within a mapped read would be a trivial process of string matching. However, relying on exact matches is fundamentally incompatible with the error profile of Oxford Nanopore Technologies (ONT).

As outlined in Chapter 2, while ONT platforms provide the continuous read lengths necessary to span the 45 kb rDNA tandem repeats, they suffer from higher error rates. More critically, the primary mode of error in nanopore sequencing is not simple SNPs, but rather indels. In the context of a k -mer-based analysis, an indel is particularly disruptive. A single uncalled or falsely inserted base within a read does not merely alter one position; it shifts the entire sequence frame. Consequently, a target rare k -mer present in the true biological molecule may be entirely destroyed or fractured in the resulting digital read, rendering standard, exact-match dictionary queries ineffective.

To successfully apply the rare k -mer filtering strategy to long-read data, the verification algorithm must transition from exact string matching to approximate string matching. Instead of demanding a perfect sequence correspondence, the tool must evaluate the presence of anchors by incorporating a defined error tolerance, commonly measured via edit distance (implemented as the Levenshtein distance). This mathematical metric quantifies the minimum number of single-character operations — insertions, deletions, or substitutions — required to transform the target reference k -mer into a substring found within the sequencing read. By allowing a specific edit distance tolerance, the algorithm can successfully recognise a sequence-specific anchor even if the nanopore basecaller has introduced artificial noise. For example, if a 31-mer anchor undergoes a single-base deletion during sequencing, an exact-match algorithm would fail to recognise it, resulting in the false rejection of a valid read. A mathematically tolerant algorithm, however, evaluates the read and identifies the region as a valid match with an edit distance of 1, thereby preserving the true alignment.

Implementing this tolerance introduces a critical computational and statistical trade-off. If the edit distance parameter is set too strictly, genuine reads containing standard sequencing errors will fail the verification step and be discarded, drastically reducing the sensitivity

of the assay. Conversely, if the error tolerance is overly permissive, the rare k-mer loses its sequence specificity. An excessive edit distance could theoretically mutate a rare anchor into a highly abundant background k-mer, allowing decoy alignments to pass the filter and reducing the overall specificity of the tool. Therefore, establishing the optimal balance of edit tolerance is critical for the success of this methodology.

3.3 Filtering Criteria

To operationalise the theoretical concepts established in the preceding sections, the post-alignment verification tool must translate biological hypotheses and error profiles into quantifiable mathematical parameters. A mapped read is not evaluated on a binary basis of a single exact match; rather, its validity is assessed through a multi-parametric scoring system. This system evaluates the sequence against distinct filtering criteria — first by defining what constitutes an "anchor" in the reference, and second by establishing the thresholds required to validate a read. The theoretical basis for each primary parameter is detailed below.

3.3.1 Defining Sequence Rarity

The foundational premise of this methodology relies on distinguishing sequence-specific anchors from the highly homologous background of the rDNA array. However, "rarity" is a relative metric that must be explicitly defined against the reference genome. Therefore, control over what frequency is accepted was needed. As the reference for rDNA is not completely resolved yet, limiting this to a single number would be too restricting and a range is preferred. Any k-mer whose frequency exceeds the defined maximal limit is statistically determined to be part of the identical tandem repeat background and should be excluded. Conversely, the minimal limit ensures the k-mer genuinely exists within the expected copy-number parameters, effectively filtering out potential sequencing artefacts. Only k-mers whose reference occurrences fall within this strict boundary are retained as locus-specific verification anchors.

3.3.2 K-mer Length

The selection of the k-mer length dictates the statistical probability of a sequence occurring by random chance. If the k-mer length is set too short, the sequence space (4^k) is insufficiently small, increasing the probability that the anchor might randomly appear in decoy loci. Conversely, if k is set too long, the probability of the sequence being entirely disrupted by Oxford Nanopore indel errors increases dramatically. Therefore, the length must be calibrated to maintain sequence specificity while surviving the error profile.

3.3.3 Sequencing Error Handling

As established in Section 2.2, strict exact-matching is incompatible with nanopore data. To accommodate this, an edit distance parameter is needed. This defines the maximum number of insertions, deletions, or substitutions allowed when comparing a reference-derived rare k-mer to the sequenced read. Since nanopore data are prone to indels, varying penalisation of different sequencing errors helps align the filtering logic with the underlying error model.

To address the unique alignment challenges of highly repetitive genomic structures — such as the tandem repeats found in ribosomal DNA (rDNA) arrays — a dynamic, density-aware tolerance mechanism is introduced. In deeply repetitive regions, verified rare k-mers are exceedingly sparse and act as critical, highly vulnerable anchors. To prevent false-positive alignments in these ambiguous zones, the algorithm demands absolute perfection from the read, enforcing a strict zero-error policy. Conversely, in regions with higher structural complexity and dense k-mer clustering (such as the flanking edges of the rDNA array), the standard user-defined edit tolerance is permitted to accommodate normal sequencing errors.

This is aimed at cases where there is little to no room for error and very high accuracy, especially in tandem repeat regions, is required.

3.3.4 Verification Confidence

Due to the elevated error rates of long-read sequencing, some true biological anchors will inevitably be mutated beyond recognition or lost

entirely, even with edit distance tolerance applied. Therefore, verifying a read requires comprehensive aggregate thresholds.

Firstly, the read must contain an absolute minimal count of rare k-mers to even be considered for further evaluation, since at least those reads without any rare k-mers are not considered trustworthy.

Secondly, the proportional presence of anchors (sufficiently intact, as defined by previous criteria) to their expected count according to the reference must also exceed a minimum percentage threshold.

4 Implementation of the Filtering Tool

The core of this thesis was the development of a computational tool to validate the rare k-mer anchoring approach. The tool needed to parse large amounts of compressed genomic data, construct an extensive k-mer index from a reference genome, and efficiently evaluate hundreds of sequence alignments.

4.1 Choice of Technology

The initial proof-of-concept for this method was prototyped in Python. Python is widely utilised in bioinformatics due to its extensive ecosystem of scientific libraries and ease of use for rapid algorithmic prototyping. Using the **pysam** library, the foundational logic of the filtering strategy — including k-mer extraction, coordinate mapping, and basic filtering — was successfully validated.

However, as the scope of the data scaled to real-world datasets containing hundreds of megabytes of genomic information, the Python implementation encountered several performance bottlenecks. These limitations are fundamentally tied to how interpreted, garbage-collected languages handle system resources. The primary constraints included:

- **Memory allocation overhead:** Processing each read in Python required allocations of new dynamic data structures on the heap, which was, on the scale of the average use case, significantly increasing the runtime.
- **I/O bottlenecks:** Abstracted file handling in higher-level languages often results in byte-to-byte system calls, unnecessarily cluttering the computation itself.

To overcome these limitations, the tool was entirely rewritten in Rust. Rust is a statically typed, ahead-of-time compiled systems programming language that guarantees memory and thread safety without the use of a garbage collector. The transition provided several critical advantages:

- **Zero-cost abstractions and buffer reuse:** Rust allows for explicit memory management. Instead of creating dynamic hash maps for

CIGAR string mapping on every read, the Rust implementation utilises the "zero-allocation" pattern. Memory buffers (contiguous flat arrays) are allocated exactly once at runtime and subsequently cleared and recycled for every alignment. This bypasses the operating system's memory allocator entirely, shifting the workload to L1 cache.

- **I/O control:** Rust exposes low-level control over file streaming. By wrapping the genomic file readers and writers in explicitly sized memory buffers, the tool drastically minimises expensive system calls, allowing the program to read data from RAM rather than spinning the storage disk.
- **Bioinformatics ecosystem:** The Rust ecosystem has matured significantly in the field of genomics. By utilising highly optimised crates, the implementation achieved C-level execution speeds while maintaining strict type safety.

The architectural shift from Python to Rust yielded a substantial increase in computational efficiency, reducing the processing time of a standard dataset tenfold. This performance gain demonstrated that the rare k-mer approach is not only theoretically sound but also implementable as a scalable tool for real-world genomic pipelines.

The implementation relies mainly on standard Rust crates (equivalent to libraries in other languages, though with some differences in their inner workings). Command-line interface (CLI) parsing was structured using **clap**, idiomatic error propagation was managed via the **anyhow** crate, and **rustc-hash** was integrated to replace Rust's default cryptographically secure hasher with a high-speed variant, used for dictionary lookups of rare k-mers. For domain-specific file operations, the **noodles** library was utilised as the core I/O engine, enabling memory-safe, strictly typed parsing of FASTA reference genomes and BGZF-compressed BAM alignments. Finally, to maximise computational throughput and eliminate CPU-bound bottlenecks during sequence evaluation, the **crossbeam-channel** crate was integrated alongside Rust's native scoped threads.

4.2 System Architecture and Workflow

The filtering tool operates in two primary phases: building a positional index of rare k-mers from the reference genome, and subsequently streaming the BAM alignments against this index.

4.2.1 Building the Rare K-mers Library

To minimise memory footprint and avoid loading unnecessary data into RAM, the indexing phase employs a two-pass strategy over the FASTA file.

Pass 1: K-mer Frequency Counting

The first pass scans the reference genome to establish global k-mer frequencies. The FASTA file is read sequentially, and every k-mer of user-defined length k is recorded, as well as the number of occurrences. Once the entire reference is parsed, the algorithm discards any k-mers that fall outside the user-defined rarity boundaries (e.g., frequencies strictly between 5 and 10), retaining only the rare k-mers.

Pass 2: Coordinate Indexing

The second pass rescans the FASTA file to map the physical coordinates of the retained rare k-mers. It cross-references each k-mer against the lookup table generated in Pass 1. If a match is found, the start and end coordinates are recorded. This data is structured into a *KmerLibrary*, grouped by chromosome name and presorted by starting position.

4.2.2 Alignment Filtering Workflow

Once the index is built, the tool processes the BAM alignments. To balance the trade-off between computational speed and absolute biological completeness, the architecture routes reads through one of two processing streams based on user configuration:

- 1) **Sequential Primary Stream:** Optimised for standard coordinate-sorted BAM files, this stream evaluates alignments sequentially. It restricts analysis to primary alignments only, ensuring maximum throughput and preserving the exact coordinate sorting of the input file.

- 2) **Grouped Secondary Stream:** Designed for comprehensive structural variant recovery, this stream requires a Name-Sorted BAM file. It buffers all alignments belonging to the same original read (QNAME). To accurately evaluate secondary or supplementary alignments — where nucleotides are physically omitted from the BAM record to save space — the algorithm dynamically borrows the full sequence from the read's primary alignment.

4.2.3 The Evaluation Engine

Regardless of the chosen stream, the core evaluation operates on thread-local, zero-allocation buffers. To maximise I/O throughput, the BAM reader and writer are wrapped in large memory buffers, preventing expensive byte-by-byte system calls. The workflow for each alignment proceeds as follows:

- **Coordinate Retrieval:** The read's 1-based reference start and end positions are extracted from the BAM record and translated to 0-based coordinates. The rare k-mers corresponding to the read location are then extracted from *KmerLibrary*.
- **CIGAR Mapping:** The CIGAR string is parsed to build a precise map translating 0-based reference coordinates to 0-based query sequence coordinates, accounting for insertions, deletions, and clipping. If a structural deletion destroys the exact start or end nucleotide of an expected k-mer, the strict boundary is lost. To maintain accurate spatial tracking, the algorithm iteratively scans inwards from the deleted edge towards the centre of the k-mer, anchoring the coordinate to the first structurally intact nucleotide it encounters.
- **Compliance Check:** For each expected k-mer, the corresponding sequence slice is extracted from the read. It is then compared with the reference rare k-mer and evaluated against user-defined parameters.
- **Output Generation:** After evaluation of all relevant rare k-mers, the read is either written to the output BAM file in case it satisfies requirements or discarded.

4.2.4 Parallelisation

To eliminate CPU-bound bottlenecks during the edit-distance computations, the Grouped Secondary Stream employs a producer-worker-writer threading model. A main producer thread dispatches read groups through crossbeam channels to a pool of scoped worker threads, which execute the evaluation engine in parallel. Because worker threads finish at slightly different times depending on the indel complexity of their assigned reads, the resulting output BAM file is emitted out of order. This prioritises absolute processing throughput, leaving downstream coordinate sorting to dedicated tools if required by subsequent pipelines.

During preliminary testing, this has yielded a 4x speedup when executed on 20 cores.

4.3 Algorithmic Details

The tool’s computational efficiency results from both architectural and algorithmic decisions. The following section details the specific optimisations implemented to process massive genomic datasets without exhausting system resources.

4.3.1 Two-Bit DNA Encoding and Rolling Bitmask

In standard FASTA and BAM files, nucleotide sequences are represented as ASCII characters (e.g., ‘A’, ‘C’, ‘G’, ‘T’), where each base occupies 1 byte (8 bits). For a standard k-mer length of $k = 31$, storing millions of k-mers as ASCII strings would quickly lead to memory exhaustion and cache thrashing.

To solve this, the tool implements a two-bit encoding algorithm. Because the standard genomic alphabet consists of only four primary bases, each base can be mathematically mapped to a 2-bit binary representation (e.g., A = 00, C = 01, G = 10, T = 11). Consequently, using a long unsigned integer (u128) gives us a maximal k-mer length of 64.

Furthermore, extracting overlapping k-mers from a reference genome using a sliding window can be computationally expensive if every window is parsed from scratch. To optimise this, the indexing algo-

rithm utilises a rolling bitmask. As the sliding window advances by one base, the tool does not re-encode the entire 31-mer. Instead, the existing 64-bit integer is shifted left by two bits, a pre-computed bitwise mask is applied to instantly erase the oldest nucleotide that just fell out of the window, and the 2-bit value of the new nucleotide is appended using a bitwise OR.

This bitwise manipulation allows the algorithm to generate the next overlapping k-mer in $O(1)$ time, thereby speeding up the linear scan of the reference genome.

4.3.2 Memory-Optimised Two-Pass Indexing

As outlined in Section 4.2, the two-pass indexing strategy is fundamentally a memory-management optimisation. Constructing the coordinate index in a single pass would require storing memory-intensive coordinate structs (positions, values, and tolerances) for every k-mer in the reference genome before determining their rarity, which would require unnecessarily more RAM.

To resolve this, the algorithm passes twice over the reference:

- 1) **Pass 1 (Frequency Filtering):** The algorithm scans the genome tracking occurrences, pairing 64-bit sequence IDs with lightweight integer counts. It yields a minimal HashSet containing only the IDs of k-mers that strictly meet the rarity thresholds.
- 2) **Pass 2 (Deferred Allocation):** Rescanning the genome, the algorithm cross-references encountered k-mers against the filtered HashSet. The larger coordinate structures are only allocated and inserted into the final FxHashMap when a verified target is found.

This strategy trades a negligible increase in I/O time (streaming the FASTA twice) for a lower memory footprint that scales strictly with the number of retained rare k-mers, rather than the entire reference genome.

4.3.3 Tolerance Evaluation

To efficiently manage biological variations and sequencing errors, the algorithm employs a cascaded tolerance evaluation for each extracted

sequence slice. Rather than immediately computing expensive edit distances, it first calculates the absolute length difference between the extracted slice and the expected k-mer. An early-termination optimisation is applied here: if the length difference multiplied by the minimum indel cost already exceeds the user-defined local tolerance, it is mathematically impossible for the sequence to pass, and the evaluation is instantly bypassed. If this check passes and the lengths are identical, the algorithm attempts a high-speed 2-bit numerical comparison. Only when errors are present — either through length discrepancies or failed exact matches — does the system fall back to the weighted Levenshtein edit distance to ultimately determine whether the sequence falls within the acceptable error tolerance.

4.4 Dynamic Tolerance

To address the unique alignment challenges of highly repetitive genomic structures — such as the tandem repeats found in ribosomal DNA (rDNA) arrays — the tool introduces a dynamic, density-aware tolerance mechanism. In deeply repetitive regions, verified rare k-mers are exceedingly sparse and act as critical, highly vulnerable anchors. To prevent false-positive alignments in these ambiguous zones, the algorithm demands absolute perfection from the read, enforcing a strict zero-error policy. Conversely, in regions with higher structural complexity and dense k-mer clustering (such as the flanking edges of the rDNA array), the standard user-defined edit tolerance is permitted to accommodate normal sequencing errors.

Algorithmically, this is achieved by calculating an "isolation distance" for every anchored k-mer, defined as the sum of the physical distances to its immediately adjacent left and right neighbours. If this isolation distance exceeds a user-defined threshold (in base pairs), the k-mer's local edit tolerance is overridden to zero. Because the index arrays are already physically presorted during the indexing phase (Section 4.3.2), fetching neighbouring coordinates requires only $O(1)$ constant-time index lookups ($i-1$ and $i+1$). This allows the tool to map context-aware, biologically logical error boundaries across the entire genome prior to filtering, with little computational overhead.

4.5 Command-Line Interface and Usage

To ensure the filtering tool is accessible and easily integrated into standard bioinformatics pipelines, the user interface was designed following standard POSIX conventions. The command-line interface (CLI) is powered by the Rust clap crate, which provides robust input validation and type-safety before any memory allocation occurs.

Rather than hardcoding biological assumptions, the tool exposes highly granular parameters, logically grouped into four operational domains.

4.5.1 I/O and Diagnostic Signatures

At its core, the tool requires only two positional arguments: the input BAM file and the reference FASTA genome. The resulting filtered data and supplementary diagnostics are managed via the following outputs:

- **-o, -output:** Specifies the destination path for the primary filtered BAM output. If omitted, the tool safely defaults to `./out.bam` in the current working directory, preventing any accidental overwriting of the original input file.
- **-bed:** Exports the exact physical coordinates of the generated rare k-mer dictionary. This allows the user to visually verify anchor density in genomic viewers (like IGV).
- **-coverage:** Calculates the inverse of the surviving read intervals, exporting a BED file of "dead zones" (regions with zero reads left after filtering). This is useful for further variant calling.

4.5.2 Dictionary Construction Boundaries

Users maintain complete control over what constitutes a "rare" k-mer in Pass 1 of the indexing phase.

1. **-len (k):** Defines the exact base-pair length of a rare k-mer (default: 31).

2. **–min and –max**: Establish the frequency boundaries for dictionary inclusion. This allows users to dynamically tune the index for specific genomic structures. (default: –min 1; –max 10)

4.5.3 Evaluation and Evolutionary Scoring

The core evaluation engine (Section 4.2.3.) is entirely configurable via the CLI, allowing researchers to tweak the strictness of the filter based on the expected error profile of their specific sequencing platform (e.g., PacBio HiFi vs. Oxford Nanopore).

- **Survival Thresholds (–inf, –pct)**: Define the absolute minimum number of sufficiently intact k-mers (dictated by other parameters) a read must contain, and the percentage of expected k-mers that must survive, respectively.
- **Error Tolerance (–tolerance, –dyn-tol)**: Sets the baseline edit distance allowed per k-mer. The –dyn-tol flag activates the context-aware spatial tolerance (Section 4.4).
- **Scoring of Different Errors (–ins-cost, –del-cost, –sub-cost)**: Allows the user to independently penalise indels versus single nucleotide polymorphisms (SNPs) during the Levenshtein distance calculation.

4.5.4 Architectural Routing

Finally, the tool exposes the –primary-only execution flag. As detailed in Section 4.2, this flag dictates the macroscopic data flow. If omitted, the tool expects a Name-Sorted BAM and boots the parallelised, memory-isolated multithreading engine to resolve secondary and supplementary alignments. If provided, it routes data through the sequential stream optimised for standard coordinate-sorted BAM files.

5 Evaluation and Results

This chapter evaluates the efficacy, accuracy, and computational efficiency of the developed filtering tool. Section 5.1 outlines the methodology, describing the characteristics of the simulated and real-world Ultra-Long datasets, the hardware and software environment, and the design of the parameter optimisation grid search. Section 5.2 then presents the algorithm’s filtering accuracy on ground-truth simulated data, establishing sensitivity and specificity metrics based on the optimised parameter values. Section 5.3 assesses the tool’s practical performance when applied to noisy, physical rDNA sequencing ONT data and Section 5.4. evaluates the tools computational efficiency. Finally, Section 5.5. discusses the results and provides recommendations to researchers using the tool.

5.1 Methodology

This section details the experimental framework designed to evaluate the performance and efficiency of the filtering algorithm. The evaluation is split into two phases: testing against rigidly controlled, simulated datasets to establish absolute mathematical baselines, followed by a qualitative evaluation using physical Oxford Nanopore sequencing data to demonstrate real-world data reduction and alignment separation. The following subsections define the characteristics of these datasets, document the hardware and software environment, outline the statistical metrics used for scoring, and describe the design of the parameter optimisation grid search.

5.1.1 Datasets

To comprehensively evaluate the filtering algorithm, the testing framework utilises three distinct categories of datasets. To establish a rigorous ground truth, the simulations utilise a diploid chromosome 14 rDNA assembly developed at the Faculty of Informatics, Masaryk University [24]. Derived from the individual PAN027 (HG06807) of the WashU pedigree, this reference contains the maternal (transcription-

Table 5.1: Reference arrays created at FI MU.

Reference Name	Origin	Array Type	Contig ID	Length (bp)
Chr14 Maternal	PAN027	Active	ptg0000005l	1,703,435
Chr14 Paternal	PAN027	Inactive	ptg0000001l	1,916,553

ally active) and paternal (transcriptionally inactive) arrays, providing a biologically accurate template for synthesising data.

The evaluation utilises the following four datasets, progressing from mathematically ideal scenarios to noisy, real-world sequencing data.

Perfect Simulated Data

Error-free synthetic reads generated directly from the reference arrays using BADREAD [25].

BADREAD is a specialized long-read sequencing simulator that deliberately models the complex and non-random artifacts inherent to Oxford Nanopore sequencing—including clustered insertions, deletions, and chimeric sequences—rather than applying idealized noise distributions.

This dataset serves as an absolute control baseline to verify the fundamental logic of the rare k -mer filtering strategy without the interference of sequencing noise.

BADREAD was run with the following parameters:

```
-quantity 200M, -length 218000,100000, -error_model random,
-identity 100,100,0, -glitches 0,0,0, -junk_reads 0,
-random_reads 0, -chimeras 0
```

Nanopore2023 Simulated Data

To evaluate the algorithm’s robustness against physical sequencing artefacts, ultra-long reads were synthesised to mimic Oxford Nanopore chemistry. Generated using the BADREAD simulator, these datasets include authentic length distributions, indels, and chimeric reads at both 97% and 99% sequence identities to mimic better and worse sequencing runs. The latest nanopore2023 model was used. This allows

for rigorous cross-validation and threshold optimisation against a known ground truth.

Badread was run with the following parameters:

```
-quantity 120M, -length 218000,50000, -error_model nanopore23,  
-qscore_model nanopore2023, -identity (99,100,0.5 for the 99%  
dataset / 97,99,1 for the 97% dataset)
```

NanoSim Simulated Data

To further validate the algorithm against empirical base-calling error profiles, a supplementary dataset was generated using NanoSim [26], which is a fast read simulator that builds a statistical profile of various sequencing characteristics. To ensure trustworthiness of the physical sequencer’s error distribution, the latest pre-trained statistical model provided by the NanoSim developers was used. This model is derived from Oxford Nanopore Kit v14 (R10 chemistry) sequencing of the human HG002 genome and basecalled with Dorado.

Nanosim was run with the following parameters:

```
-x 60, -t 4, -fastq, -hp, -k 5
```

Sequencing Data

A real-world Ultra-Long Oxford Nanopore dataset. Because the absolute biological ground truth of physical reads mapping to modelled rDNA regions cannot be definitively established, this dataset is not used for statistical accuracy metrics. Instead, it is utilised to evaluate the algorithm’s practical data reduction capabilities and to perform qualitative visual validation of the filtered alignments.

Overview of the Datasets

These simulated reads were then combined into one file and mapped onto both maternal and paternal arrays. Due to the structural similarity and repetitiveness of the arrays, the vast majority of the generated reads from maternal array mapped successfully onto the paternal and vice versa (see table below), creating datasets with sequences originating from the genomic regions (target reads) and similar sequences, but from another array (decoy reads). This is precisely what happens

Table 5.2: Overview of all Datasets

Dataset	Type	Read Count	Avg. Length	N50	Total Yield
Perf. (Mat.)	badread	983	203 kb	241 kb	200 Mb
Perf. (Pat.)	badread	975	205 kb	244 kb	200 Mb
99% (Mat.)	badread	586	205 kb	222 kb	120 Mb
99% (Pat.)	badread	575	208 kb	223 kb	120 Mb
97% (Mat.)	badread	604	199 kb	216 kb	120 Mb
97% (Pat.)	badread	582	206 kb	223 kb	120 Mb
NanoSim (Mat.)	NanoSim	6,007	16.6 kb	28.7 kb	100 Mb
NanoSim (Pat.)	NanoSim	6,756	16.4 kb	28.8 kb	111 Mb
Sequenced	ONT	129	218kb	228kb	187Mb

when assembling/mapping to a rDNA reference, which makes this an ideal test case for the rare k-mer filtering approach.

Further in the thesis, the combined reads mapped onto either only maternal or only paternal array are spoken of as simply “maternal” and “paternal”.

5.1.2 Hardware and Software

To ensure reproducibility and establish an accurate baseline for computational efficiency, all processing, simulations, and parameter optimisation grid searches were executed on a dedicated Linux workstation. The system was running Ubuntu 24.04 LTS and equipped with an Intel Core i5-14600K processor and 32 GB of DDR5 RAM.

To verify cross-platform compatibility, the program was additionally compiled and validated on a macOS environment (Apple MacBook Pro 14), demonstrating execution across different UNIX-like operating systems.

The primary algorithm, bambam, was compiled using the stable Rust toolchain in release mode. Supporting operations, including read mapping and alignment sorting, were handled by minimap2 [6] and samtools [4], respectively. Synthetic dataset generation was executed using badread [25] and NanoSim [26]. Finally, the parameter optimisation script was implemented in Python.

5.1.3 Evaluation Metrics

To quantitatively assess the performance of the filtering algorithm, the read separation process is modelled as a binary classification problem. The primary objective is to correctly identify and retain reads originating from the target rDNA array while accurately discarding reads from decoy arrays or background genomic noise. The classification outcomes are defined as follows:

- True Positives (TP): Target reads correctly retained by the filter.
- False Positives (FP): Decoy reads incorrectly retained by the filter.
- True Negatives (TN): Decoy reads correctly discarded by the filter.

- False Negatives (FN): Target reads incorrectly discarded by the filter.

To systematically evaluate the algorithm's accuracy, three fundamental statistical metrics are derived from these outcomes:

Sensitivity measures the algorithm's ability to capture the desired target reads, which ensures minimal loss of valuable sequencing data:

$$\text{Sensitivity} = \frac{TP}{TP + FN}$$

Specificity measures the algorithm's ability to successfully reject decoy reads, which ensures that the filtered dataset is not contaminated with structural noise:

$$\text{Specificity} = \frac{TN}{TN + FP}$$

To establish a single, robust optimisation target for the automated parameter grid search, the F_1 -score is utilised. This ensures a balanced performance evaluation and penalises parameter combinations that achieve high data retention at the expense of decoy contamination, or vice versa:

$$F_1\text{-score} = 2 \times \frac{\text{Precision} \times \text{Sensitivity}}{\text{Precision} + \text{Sensitivity}}$$

5.1.4 Finding Optimal Parameters

The efficacy of the rare k -mer approach is dependent on several algorithmic thresholds. Because physical sequencing reads exhibit variable lengths and error distributions, manually selecting these thresholds risks introducing human bias and suboptimal filtering performance. To systematically identify the optimal configuration, a comprehensive parameter optimisation grid search was conducted.

The grid search evaluated a matrix of approximately 6000 unique parameter combinations for each dataset. The search space encompassed the following core variables:

- **k -mer Length (-len):** The exact nucleotide length of the rare anchors. The search evaluated lengths **21, 31, 41, 51, and 61** to find the bal-

ance between sequence specificity and susceptibility to base-calling errors.

- **K-mer Count (-min, -max):** Defines the allowed frequency bounds for rare k -mers within the reference. Minimum bounds: **1, 3, 5**; maximum bounds: **10 and 15**.
- **Coverage/Match Thresholds (-pct):** The minimum percentage of the read length—or expected anchor density—required to confidently classify a read as a target. Values: **70, 80, 85, 90 and 95**.
- **Error Penalties and Distance (-sub-cost, -tolerance):** Penalisation of different kinds of sequencing errors and the allowable genomic distance between corresponding k -mer anchors. Evaluated values for genomic distance were **0, 1, 2, 3 and 5**. Penalty for substitution was tested at **1** (standard) and **2**.
- **Dynamic Tolerance:** Enforcing no distance tolerance for isolated k -mers. Either **on** or **off**.

The grid search executed every possible permutation, logging the True Positives, False Positives, and resulting F_1 -score for each run.

5.2 Filtering accuracy on Simulated Data

This chapter presents the results of the grid search described in Section 5.1.4.

5.2.1 Perfect Reads

Prior to evaluating the algorithm against simulated sequencer noise, a mathematical baseline was established using the error-free synthetic read datasets. Because these datasets lack structural chimeras and base-calling artifacts, they serve strictly as a control to verify the theoretical soundness of the rare k -mer strategy.

As this dataset served as a basic validation of the fundamental logic, the grid search conducted did not include as many options as the following, since, without the sequencing noise, there was not as much room for error.

Table 5.3: Parameters used in the conducted grid search.

K-mer length	Min	Max	Pct	Tolerance	Inf	Dyn-tol
21, 31, 41, 51, 61	3, 5	10, 15	98, 100	0, 1	1, 3	FALSE

Table 5.4: Statistics of perfect read datasets pre-filtering.

Metric	Maternal	Paternal
Absolute Targets in TXT	983	975
Absolute Decoys in TXT	975	983
Total Reads in BAM	1958	1958
True Positives (TP)	983	975
False Positives (FP)	975	983
True Negatives (TN)	0	0
False Negatives (FN)	0	0
Sensitivity (Recall)	1	1
Specificity	0	0
Precision	0.502	0.498
F1-Score	0.6685	0.6648

The grid search was conducted with the parameters shown in Table 5.3.

The metrics for both maternal and paternal dataset versions before filtering are in Table 5.4.

As expected, the algorithm achieved near-perfect separation, yielding F_1 -scores exceeding 0.99 for both maternal and paternal target classifications. The fractional margin of error (representing negligible False Positives or False Negatives) is attributed not to algorithmic failure, but to inherent biological homology; rare reads drawn from highly conserved sub-regions of the arrays may lack sufficient sequence divergence to establish diagnostic anchors.

Table 5.5: Best results from a grid search on perfect data (sorted by highest F1).

Target array	TP	FN	TN	FP	Sensitivity	Specificity	F1
Maternal	982	1	975	0	0.9990	1.000	0.9995
Paternal	971	4	977	6	0.9959	0.9939	0.9949

The Table 5.5 shows the results of the best runs for both maternal and paternal arrays.

Having confirmed the fundamental filtering logic, the grid search optimization was subsequently applied to the error-prone simulations.

5.2.2 99% Reads

Following the baseline verification, the algorithm was evaluated against the 99% identity badread dataset, introducing realistic Oxford Nanopore base-calling errors, indels, and structural chimeras. The objective was to determine the algorithm’s robustness to sequencer noise and identify the parameter thresholds required to maximise data retention.

The metrics for both maternal and paternal dataset versions before filtering are in Table 5.6.

The grid search identified an optimal parameter configuration yielding a maximum peak F_1 -score of 0.9599 for the maternal target array and F_1 -score of 0.9447 for the paternal target array. The complete classification metrics for this optimal configuration are detailed in Table 5.7.

5.2.3 97% Reads

To stress-test the algorithm’s boundaries against higher frequencies of sequencing errors, the evaluation was repeated using the 97% identity badread dataset. With an error rate three times higher than the 99% simulation, the probability of indels disrupting the exact rare k-mer anchors increases significantly, forcing the algorithm to rely more heavily on dynamic spatial tolerance.

The metrics for both maternal and paternal dataset versions before filtering are in Table 5.10.

Table 5.6: Statistics of 99% reads datasets pre-filtering.

Metric	Maternal	Paternal
Absolute Targets in TXT	586	575
Absolute Decoys in TXT	575	586
Total Reads in BAM	1144	1144
True Positives (TP)	576	568
False Positives (FP)	568	576
True Negatives (TN)	7	10
False Negatives (FN)	10	7
Sensitivity (Recall)	0.9829	0.9878
Specificity	0.0122	0.0171
Precision	0.5035	0.4965
F1-Score	0.6659	0.6608

Table 5.7: Best results from a grid search on 99% data (sorted by highest F1).

Target array	TP	FN	TN	FP	Sensitivity	Specificity	F1 Score
Maternal	562	24	552	23	0.9590	0.9600	0.9599
Paternal	564	11	531	55	0.9809	0.9061	0.9447

Table 5.8: Best scoring parameters on 99% Maternal dataset (sorted by highest F1).

Kmer	Min	Max	Pct	Tolerance	Inf	Dyn_Tol	SNPs	F1 Score
31	3	15	95	5	3	TRUE	2	0.9599
31	5	15	95	5	3	TRUE	2	0.9597
31	5	15	95	5	1	TRUE	2	0.9597

5. EVALUATION AND RESULTS

Table 5.9: Best scoring parameters on 99% Paternal dataset (sorted by highest F1).

Kmer	Min	Max	Pct	Tolerance	Inf	Dyn_Tol	SNPs	F1_Score
21	5	15	95	5	1	TRUE	2	0.9447
21	5	15	95	5	1	TRUE	1	0.9433
21	5	15	95	5	3	TRUE	2	0.9430

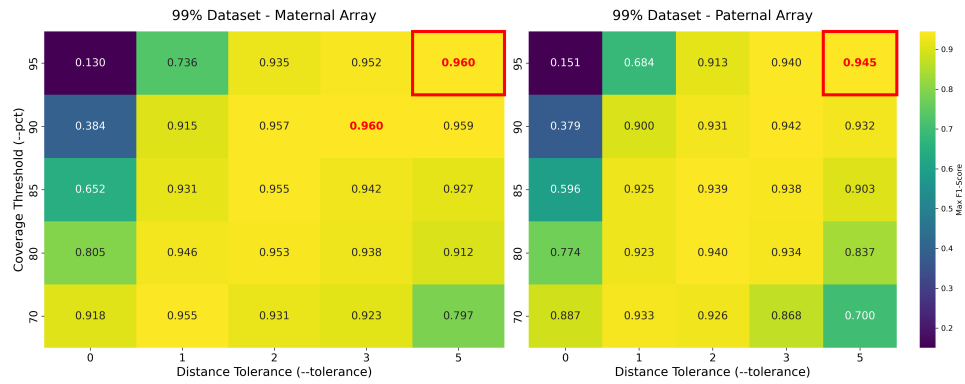


Figure 5.1: Heatmap showcasing relationship between percentage of intact k-mers and distance tolerance

5. EVALUATION AND RESULTS

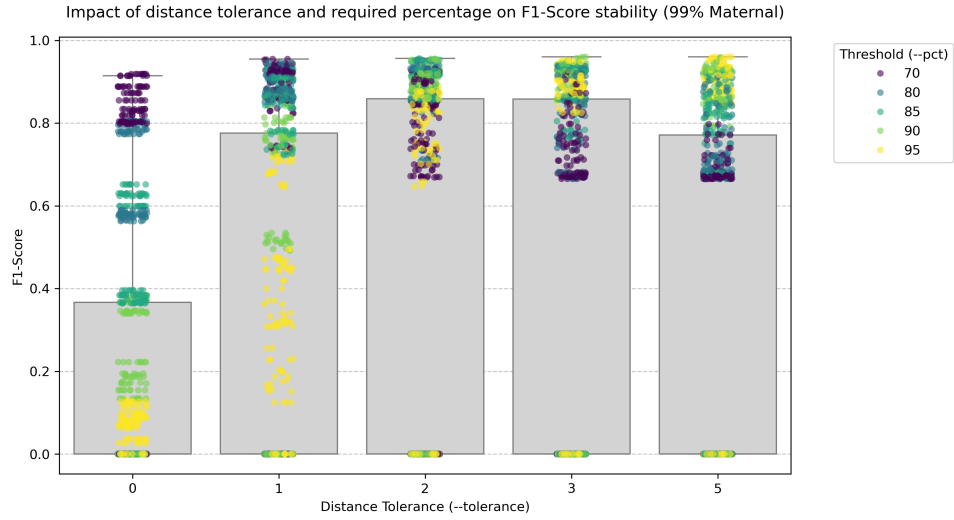


Figure 5.2: Impact of distance and percentage on F1 shows inverse relationship, suggesting that room for error is necessary.

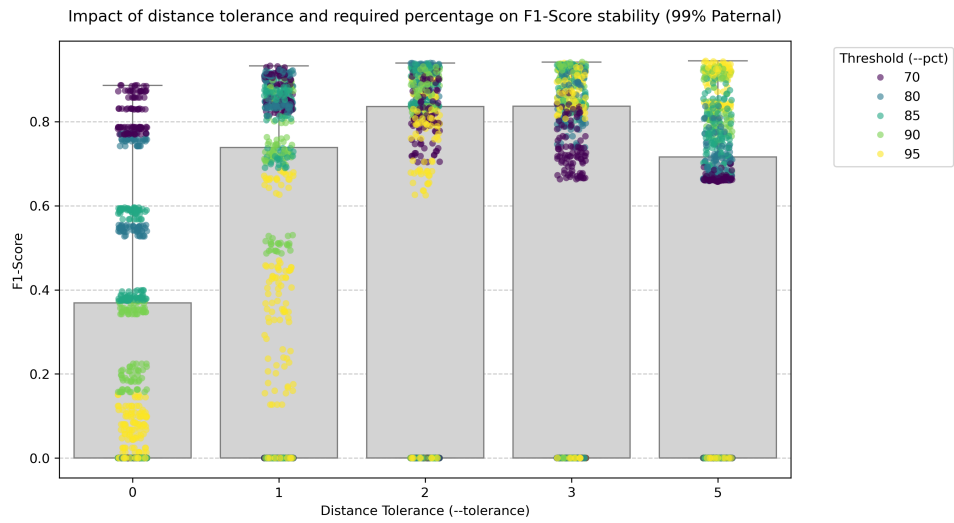


Figure 5.3: Impact of distance and percentage on F1 shows inverse relationship, suggesting that room for error is necessary (for completeness).

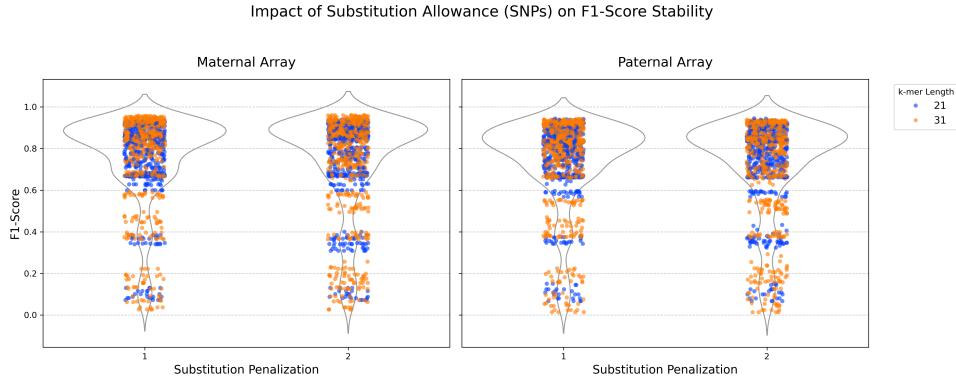


Figure 5.4: Different penalisation of substitution errors shows little to no effect on F1.

Table 5.10: Statistics of 97% reads datasets pre-filtering.

Metric	Maternal	Paternal
Absolute Targets in TXT	604	582
Absolute Decoys in TXT	582	604
Total Reads in BAM	1162	1162
True Positives (TP)	590	572
False Positives (FP)	572	590
True Negatives (TN)	10	14
False Negatives (FN)	14	10
Sensitivity (Recall)	0.9768	0.9828
Specificity	0.0172	0.0232
Precision	0.5077	0.4923
F1-Score	0.6682	0.656

Table 5.11: Best results from a grid search on 97% data (sorted by highest F1).

Target array	TP	FN	TN	FP	Sensitivity	Specificity	F1 Score
Maternal	565	39	550	32	0.9354	0.9450	0.9409
Paternal	552	30	531	73	0.9485	0.8791	0.9147

Table 5.12: Best scoring parameters on 97% Maternal dataset (sorted by highest F1).

Kmer	Min	Max	Pct	Tolerance	Inf	Dyn_Tol	SNPs	F1_Score
31	5	15	90	5	3	TRUE	1	0.9409
31	5	15	90	5	1	TRUE	1	0.9401
31	5	15	90	5	1	TRUE	2	0.9313

The grid search identified a maximum F_1 -score of 0.9409 for the maternal array and F_1 -score of 0.9147 for the paternal array. While this represents an expected decrease in absolute classification accuracy compared to the 99% dataset, the algorithm successfully maintained viable read separation without experiencing significant sensitivity failure. Complete metrics are reported in Table 5.11.

5.2.4 NanoSim Reads

Having established the biological realism of the NanoSim dataset, the subsequent evaluation serves as the more stringent theoretical stress test of the algorithm. By introducing authentic, context-dependent base-calling errors and homopolymer biases, these results provide the primary indicator of the tool's readiness for physical sequencing data.

Table 5.13: Best scoring parameters on 97% Paternal dataset (sorted by highest F1).

Kmer	Min	Max	Pct	Tolerance	Inf	Dyn_Tol	SNPs	F1_Score
31	5	15	80	3	1	FALSE	1	0.9147
31	5	15	90	5	1	FALSE	1	0.9143
31	5	15	90	5	1	TRUE	1	0.9142

5. EVALUATION AND RESULTS

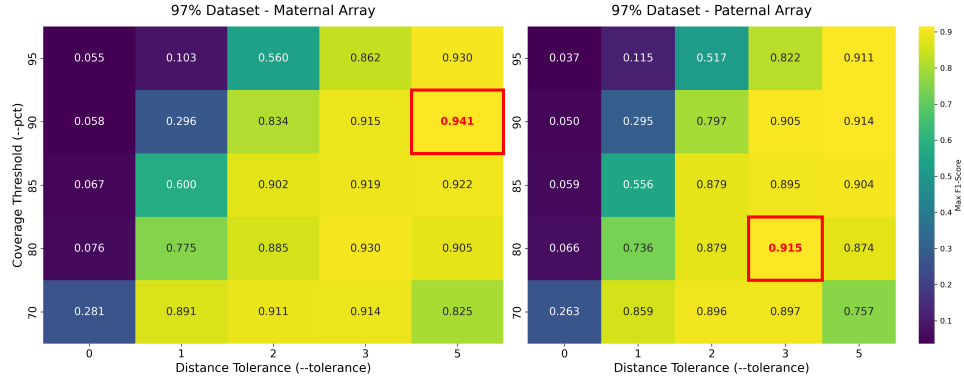


Figure 5.5: Heatmap showcasing relationship between percentage of intact k-mers and distance tolerance (97% dataset)

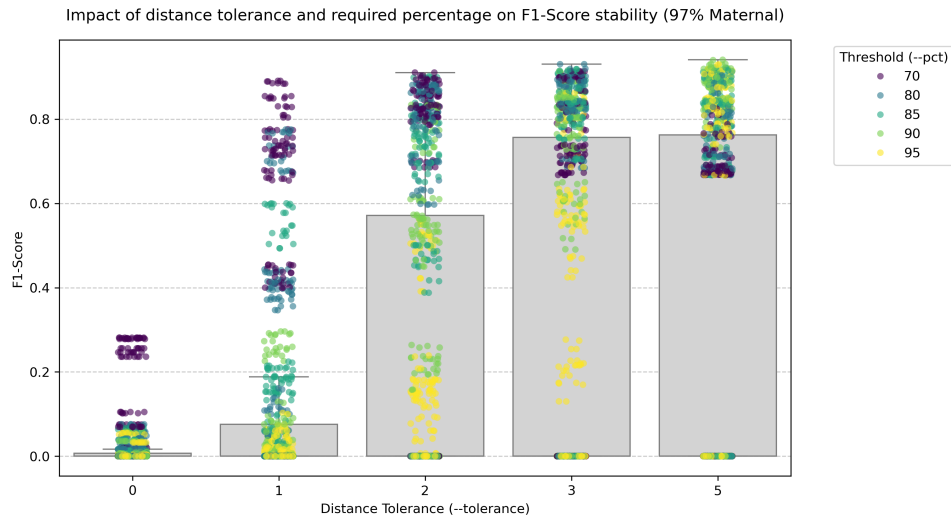


Figure 5.6: Impact of distance and percentage on F1 shows inverse relationship, suggesting that room for error is necessary (97% dataset).

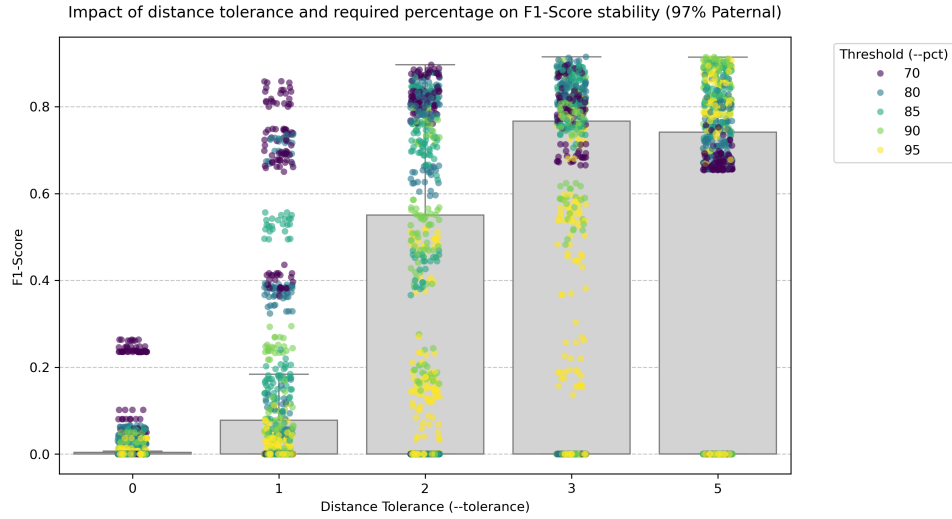


Figure 5.7: Impact of distance and percentage on F1 shows inverse relationship, suggesting that room for error is necessary (for completeness, 97% dataset).

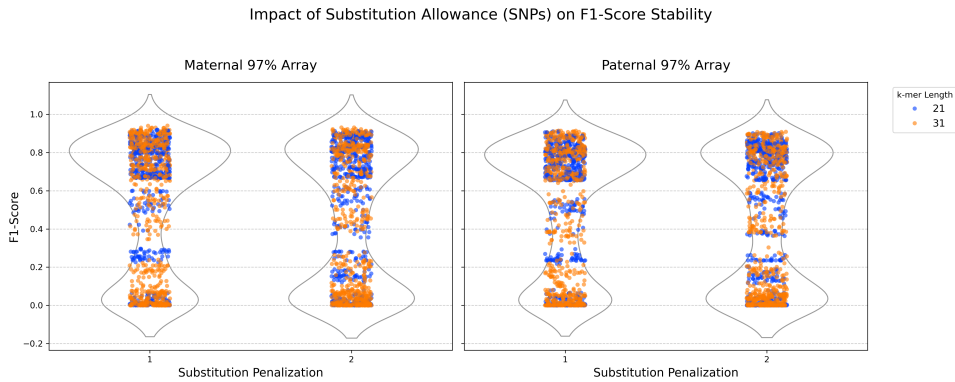


Figure 5.8: Different penalisation of substitution errors shows little to no effect on F1 (97% dataset). Substitution cost 2 shows only slightly higher prevalence of worse F1.

Table 5.14: Statistics of NanoSim reads datasets pre-filtering.

Metric	Maternal	Paternal
Absolute Targets in TXT	6007	6756
Absolute Decoys in TXT	6756	6007
Total Reads in BAM	12736	12736
True Positives (TP)	5994	6742
False Positives (FP)	6742	5994
True Negatives (TN)	14	13
False Negatives (FN)	13	14
Sensitivity (Recall)	0.9978	0.9979
Specificity	0.0021	0.0022
Precision	0.4706	0.5294
F1-Score	0.6396	0.6918

Table 5.15: Best results from a grid search on NanoSim data (sorted by highest F1).

Target array	TP	FN	TN	FP	Sensitivity	Specificity	F1 Score
Maternal	5394	613	4863	1893	0.8980	0.7198	0.8115
Paternal	5723	1033	4464	1543	0.8471	0.7431	0.8163

The metrics for both maternal and paternal dataset versions before filtering are in Table 5.14.

As summarised in Tables 5.16 and 5.17, the algorithm maintains high classification stability across both alleles, achieving peak F_1 -scores of 0.8115 for the maternal array and 0.8163 for the paternal array.

5.3 Qualitative Evaluation on Sequencing Data

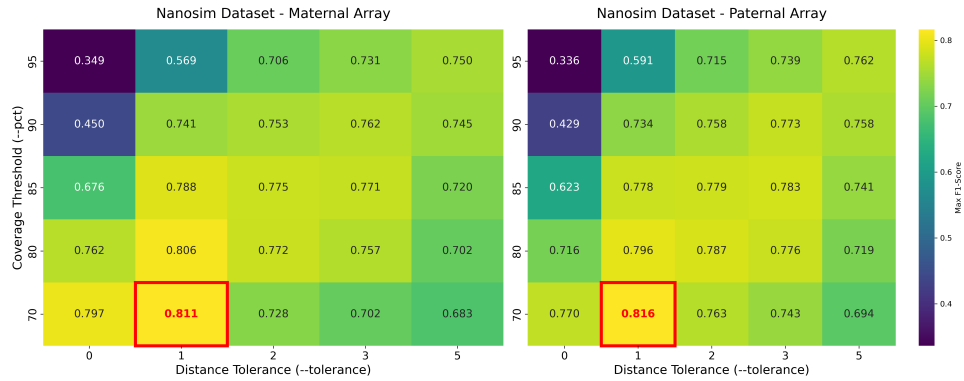
To evaluate the algorithm's performance in a real-world production environment, the optimal parameters identified during the simulation

Table 5.16: Best scoring parameters on NanoSim Maternal dataset (sorted by highest F1).

Kmer	Min	Max	Pct	Tolerance	Inf	Dyn_Tol	SNPs	F1_Score
31	1	15	70	1	1	TRUE	2	0.8115
31	1	15	70	1	3	TRUE	2	0.8114
31	1	15	70	1	1	FALSE	2	0.8112

Table 5.17: Best scoring parameters on NanoSim Paternal dataset (sorted by highest F1).

Kmer	Min	Max	Pct	Tolerance	Inf	Dyn_Tol	SNPs	F1_Score
31	1	15	70	1	3	FALSE	2	0.8163
31	1	15	70	1	3	TRUE	2	0.8159
31	1	15	70	1	1	FALSE	2	0.8154

**Figure 5.9:** Heatmap showcasing relationship between percentage of intact k-mers and distance tolerance (NanoSim dataset). Noisier reads benefit from allowing lower percentage while lowering tolerance.

5. EVALUATION AND RESULTS

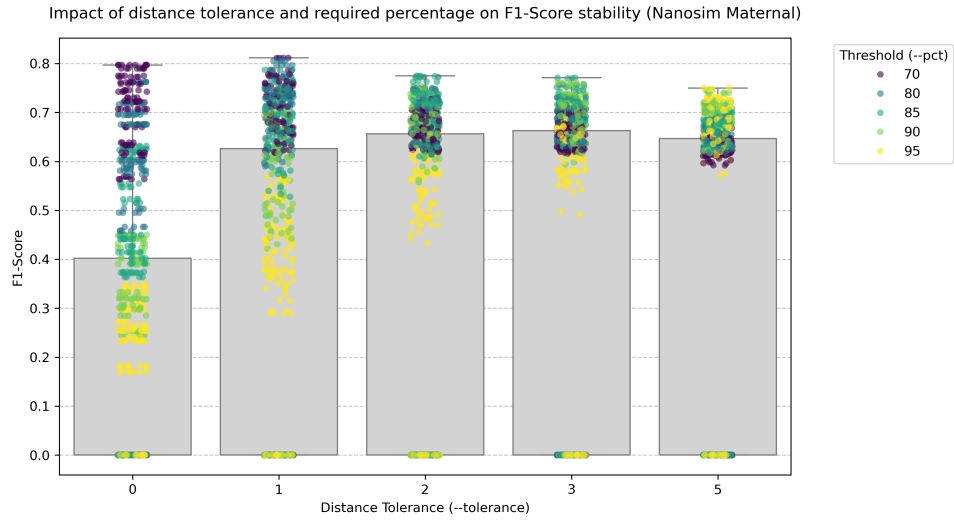


Figure 5.10: Impact of distance and percentage on F1. The differences between different distance values are less pronounced (NanoSim dataset).

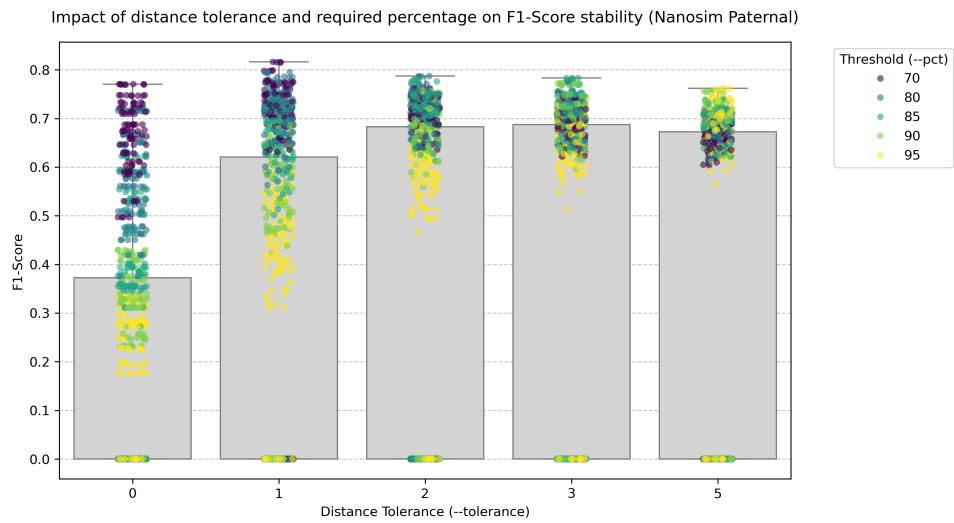


Figure 5.11: Impact of distance and percentage on F1. The differences between different distance values are less pronounced (for completeness, NanoSim dataset).

5. EVALUATION AND RESULTS

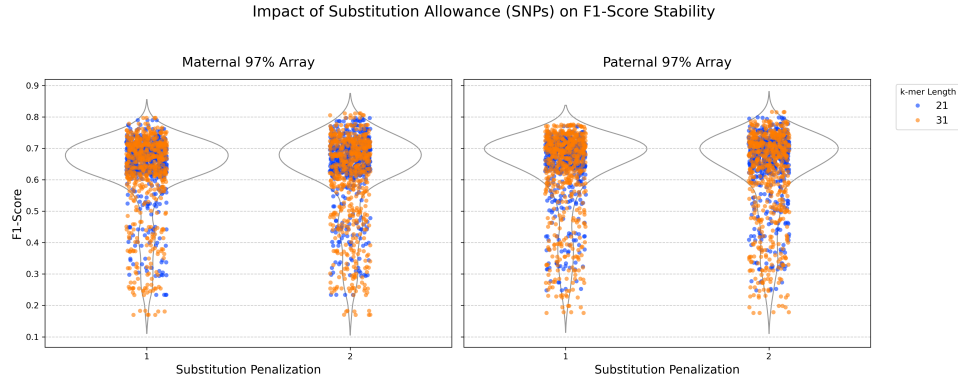


Figure 5.12: Different penalisation of substitution errors and its effect on F1 (NanoSim dataset). Substitution cost 2 shows worse F1 scores on noisier data.

Table 5.18: Parameter choices tested against sequencing data.

Dataset	Kmer	Min	Max	Pct	Tolerance	Inf	Dyn_Tol	SNPs
Maternal 99	31	3	15	95	5	3	TRUE	2
Paternal 99	21	5	15	95	5	1	TRUE	2

phase (Section 5.2) were applied to a Ultra-Long Oxford Nanopore dataset. This evaluation focuses on the tool’s ability to extract high-confidence ribosomal DNA (rDNA) reads from a complex genomic background where the absolute ground truth is not mathematically defined.

5.3.1 Experimental Setup and Parameter Selection

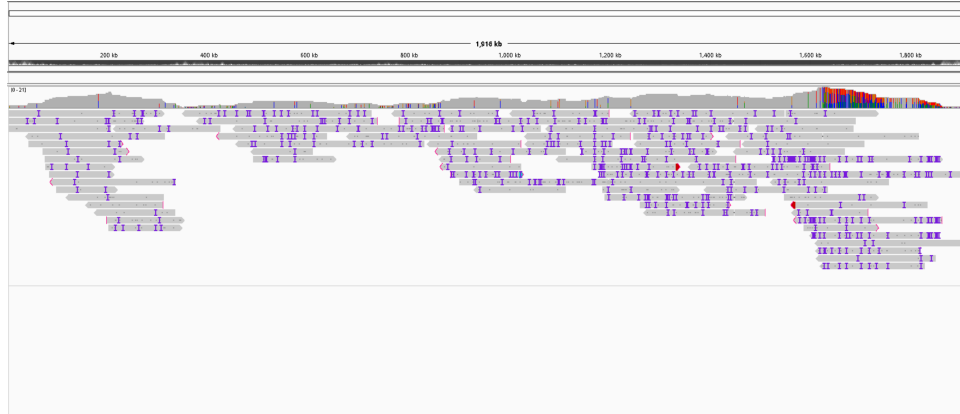
The filtering was executed using two distinct configurations derived from the peak performance zones of the previous grid searches, see Table 5.18.

5.3.2 Data Reduction

The primary objective of the qualitative test was to observe the efficiency of the rare k-mer strategy in reducing data volume while

Table 5.19: Resulting read counts after filtering.

Dataset	Reads Before Filtering	Reads After 99% Maternal Config.	Reads After 99% Paternal Config.
Maternal	46	46	46
Paternal	83	71	79

**Figure 5.13:** Screenshot of IGV showing retained reads after filtering with optimal configuration (Maternal array).

retaining target coverage. The results for the primary maternal and paternal arrays are summarised in Table 5.19.

5.3.3 IGV

The filtered alignments were loaded into the Integrative Genomics Viewer (IGV) [27] to inspect the continuity and depth of the rDNA arrays.

5.4 Computational Efficiency

To evaluate the computational efficiency and scaling behavior of bam-bam, the computationally demanding parameters were tested. The benchmarking measured memory consumption and User CPU Time for processing speed. The tests were performed using a maternal chromosome 14 reference FASTA and the 99% identity BAM file mapped onto the reference. The primary variables investigated were the k-mer



Figure 5.14: Screenshot of IGV showing retained reads after filtering with optimal configuration (Paternal array).

length ($k \in \{21, 31, 41, 51, 61\}$) and the maximum allowed edit distance or tolerance ($d \in \{0, 1, 2, 3, 5\}$). All benchmarks were restricted to the sequential branch.

Across all 25 tested parameter combinations of length and tolerance, the peak RAM utilization remained strictly bounded between 30.6 MB and 31.2 MB, as shown in the figure 5.15.

Notably, increasing the k-mer length from 21 to 61—which significantly increases the bit-width required to store the genomic sequences—did not result in a linear expansion of memory usage. Similarly, relaxing the edit distance ($d = 5$) had no measurable impact on RAM. While the memory footprint remained static, the processing time exhibited strong dependence on both k-mer length and edit distance. As illustrated in Figure 5.16, strict exact matching ($d = 0$) operates in highly optimised linear time relative to the input data, completing the dataset in approximately 4.0 to 5.2 seconds regardless of the chosen k-mer length. However, introducing even a single permissible edit distance ($d = 1$) triggers a sharp inflexion point in processing time.

To contextualise the computational cost of user configurations, Table 5.20 normalises the processing time against the exact-match baseline for each k-mer length.

5. EVALUATION AND RESULTS

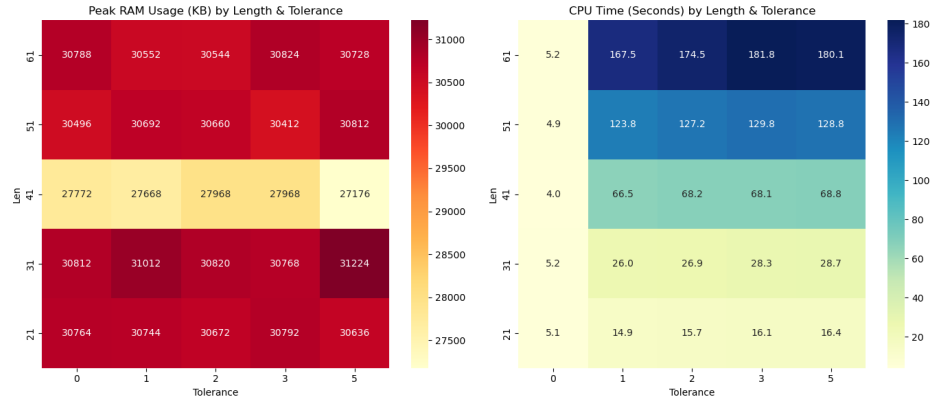


Figure 5.15: Heatmaps showcasing RAM consumption and CPU time based on parameter choice.

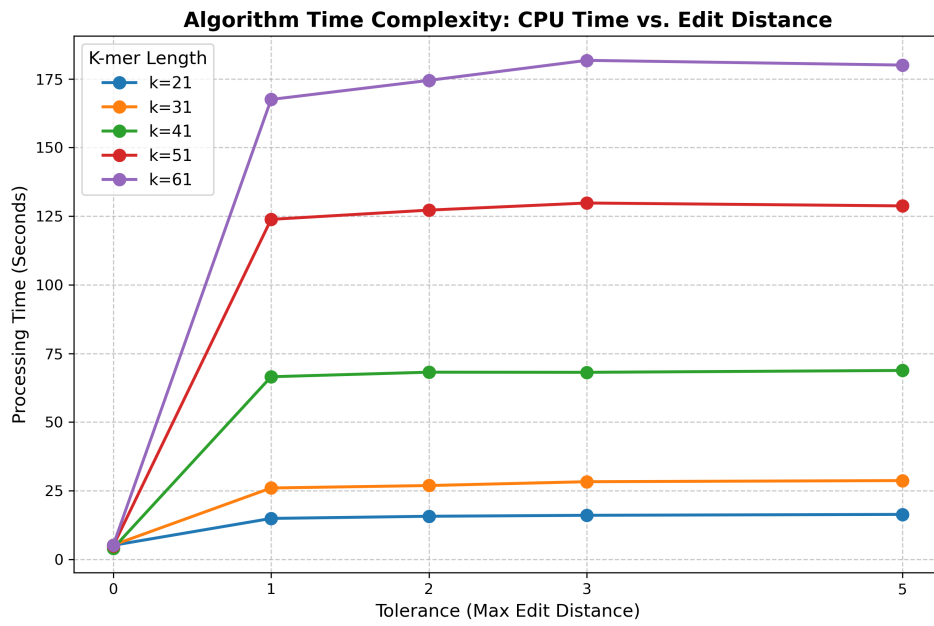


Figure 5.16: Relationship of distance tolerance to processing time (seconds) for different k-mer lengths.

Table 5.20: Table showing relative computational cost for different combinations of k-mer length and distance tolerance.

K-mer Length	Tol=0 (Exact Match)	Tol=1	Tol=2	Tol=3	Tol=5
61	5.17s (1.0x)	32.4x	33.7x	35.2x	34.8x
51	4.93s (1.0x)	25.1x	25.8x	26.3x	26.1x
41	4.05s (1.0x)	16.4x	16.8x	16.8x	17.0x
31	5.22s (1.0x)	5.0x	5.2x	5.4x	5.5x
21	5.14s (1.0x)	2.9x	3.1x	3.1x	3.2x

5.5 Discussion

The evaluation of the **Bambam** filtering tool across a spectrum of simulated and physical datasets confirms the viability, robustness, and computational efficiency of the rare k -mer filtering strategy. The obtained results provide clear insights into the algorithm’s strengths and its behaviour under varying levels of sequencing noise.

5.5.1 Algorithm Efficacy and Robustness to Noise

The fundamental logic of the rare k -mer intersection strategy was definitively validated by the perfect simulated dataset, which achieved near-perfect separation with F_1 -scores exceeding 0.99 for both maternal and paternal arrays. The negligible classification errors observed in this ideal scenario are attributed to inherent biological homology rather than algorithmic flaws; rare reads drawn from highly conserved sub-regions of the arrays simply lack sufficient sequence divergence to establish diagnostic anchors. When subjected to physical sequencing artefacts using the badread models, the algorithm demonstrated strong resilience. At a 99% identity rate, the tool maintained peak F_1 -scores of 0.9599 (maternal) and 0.9447 (paternal). Even when stress-tested against a 97% identity dataset—where the higher error rate significantly disrupts exact k -mer anchors—the algorithm gracefully degraded, maintaining F_1 -scores above 0.91 without experiencing severe sensitivity failure. The NanoSim dataset provided the most stringent theoretical stress test by introducing authentic, context-dependent base-calling errors and homopolymer biases representative of Oxford Nanopore R10.4.1 chemistry. Under these challenging conditions, the

F_1 -scores stabilised at approximately 0.811 and 0.816. While lower than the badread simulations, these metrics indicate that the tool remains capable of viable classification even when faced with complex, non-random sequencer noise. Furthermore, qualitative evaluations on physical Ultra-Long ONT data demonstrated the tool’s ability to retain reads when faced with the reality of sequencing artefacts, as the mapped reads used in this thesis were already of high confidence.

5.5.2 Parameter Influence and Interaction

The comprehensive grid search provided critical insights into how individual algorithmic parameters govern filtering performance. The data clearly indicates that the primary drivers of classification accuracy are the required percentage of intact k-mers (pct) and the allowed edit distance (tolerance). By contrast, the absolute minimum count of k-mers required per read (inf) exhibited no significant influence on the final outcomes across the evaluated datasets.

Furthermore, the results highlight a strict inverse relationship between pct and tolerance. As the required percentage of valid k-mers increases, achieving optimal separation becomes heavily dependent on allowing a non-zero edit distance. This confirms that enforcing strict exact matching across a high percentage of anchors is biologically unrealistic for long-read data; some degree of local error tolerance is strictly necessary to accommodate inevitable sequencing artefacts.

A distinct artefact was also observed in the performance distributions for the 99% identity dataset, where the F_1 -scores exhibited visible quantisation at 0 edit distance tolerance. Analysis of the grid search data confirmed that this quantisation does not correlate with any specific algorithmic parameter variation. It is highly probable that it is an artefact of the simulated data itself, arising either from the discrete algorithmic behaviours of the underlying badread Nanopore2023 error model or from the inherent, highly structured repetitiveness of the target rDNA arrays.

Finally, the evaluation established a functional upper bound for k-mer lengths when dealing with noisy long-read data. Lengths exceeding 31 base pairs—specifically $k \in \{41, 51, 61\}$ —resulted in a complete collapse of sensitivity, effectively yielding zero target retention (and were therefore excluded from the primary performance

figures). Because longer sequence windows possess a mathematically higher probability of encompassing random sequencing errors, the baseline edit distances evaluated were possibly insufficient to rescue them. Future iterations of this approach may benefit from dynamically scaling the allowed tolerance proportionally to the chosen k-mer length, which could potentially restore the viability of longer, highly specific anchors.

An additional observation derived from the grid search concerns the context-aware dynamic tolerance parameter (`-dyn-tol`). Despite its algorithmic importance for resolving repetitive sequences, activating this spatial bottleneck yielded no statistically significant impact on the macroscopic classification metrics. Across the heavily mutated NanoSim datasets, F_1 -scores remained functionally identical whether dynamic tolerance was enabled or disabled. However, this behaviour aligns precisely with the parameter's intended biological design. Dynamic tolerance was not engineered to be a primary driver of global read separation (differentiating target reads from decoys); rather, it functions as a highly localised quality-control mechanism. Its sole purpose is to strictly enforce zero-error anchor matching within deeply repetitive tandem arrays—regions where sparse k-mers act as highly vulnerable anchors and must be trusted absolutely. Consequently, while the feature does not meaningfully alter global sensitivity or specificity metrics, it provides researchers with a critical tool to strictly enforce alignment integrity within the most challenging, biologically repetitive genomic zones.

5.5.3 Biological recommendations

Synthesising results from comprehensive evaluations across varied datasets, this chapter provides concrete parameter recommendations to guide others using bambam. Optimal configurations are tightly coupled with the noise profile and structural characteristics of the input data, requiring a strategic balance between classification sensitivity and specificity.

For datasets generated with modern sequencing chemistries and basecallers that exhibit low error rates (typically $<2\%$) and robust global alignments with minimal indels, the algorithm can prioritise

Table 5.21: Recommended "strict" parameter configuration.

Kmer	Min	Max	Pct	Tolerance	Inf	Dyn_Tol	SNPs
31	3	15	95	5	3	TRUE	2

Table 5.22: Recommended parameter configuration for noisier datasets.

Kmer	Min	Max	Pct	Tolerance	Inf	Dyn_Tol	SNPs
31	1	15	70	1	1	TRUE	2

high specificity to avoid false positives from homologous decoy regions.

The recommended "Strict" configuration for high-fidelity data is presented in Table 5.21.

For noisier datasets or potentially shorter reads, a more permissive strategy is necessary to maintain acceptable sensitivity, accepting slightly reduced specificity to prevent excessive target read rejection. These optimal parameters are derived from simulations on challenging noise profiles.

6 Conclusion

The primary objective of this thesis was to address the challenge of false-positive long-read alignments within highly homologous rDNA loci. Recognising the limitations of standard greedy alignment heuristics when processing highly repetitive and error-prone long-read data, this work explored the "rare k-mer" approach as a robust, post-alignment verification strategy.

To realize this, an efficient command-line tool was implemented in Rust. The tool parses standard BAM files and filters alignments based on the positional compliance of sequence-specific rare k-mers. Based on the user preferences, it calculates edit distance to allow for some tolerance. The algorithm successfully accounted for the high indel error profiles inherent to long-read sequencing technologies, distinguishing true anchoring k-mers from sequencing noise.

The experimental evaluation benchmarked this tool against simulated and performed qualitative analysis on real rDNA data. The results proved this filtering strategy to be a viable approach to post-alignment verification.

Furthermore, the transition from the initial Python prototype to the final Rust implementation yielded significant performance improvements, confirming the tool's viability for efficiently processing larger genomic datasets.

Ultimately, this thesis demonstrates that utilising rare k-mers is an effective method for distinguishing true target alignments from decoys in complex genomic regions. The developed tool provides researchers with a functional mechanism for confident read filtering. Future work could expand upon this foundation by implementing k-mer weighting strategies—assigning higher significance to statistically rarer k-mers—or by integrating the filtering logic directly into the mapping algorithms themselves, further advancing our ability to analyse the remaining mysteries of the human genome.

7 Appendices

Data attachment

The folder `source_code` contains the Rust source code for the tool.

Folder `data` contains the following subfolders:

1. `perfect` - containing the perfect simulated reads as well as grid search results
2. `99` - containing the 99% simulated reads as well as grid search results
3. `97` - containing the 97% simulated reads as well as grid search results
4. `nanosim` - containing NanoSim simulated reads as well as grid search results
5. `ONT` - containing sequencing data

The folder `scripts` contains the python scripts used during the development of the tool.

Bibliography

1. *The Complete Sequence of a Human Genome* [<https://www.science.org/doi/10.1126/science.abj6987>]. [N.d.]. [visited on 2026-03-30]. Available from doi: 10.1126/science.abj6987.
2. LU, Hengyun; GIORDANO, Francesca; NING, Zemin. Oxford Nanopore MinION Sequencing and Genome Assembly. *Genomics, Proteomics & Bioinformatics* [online]. 2016, vol. 14, no. 5, pp. 265–279 [visited on 2026-03-30]. ISSN 1672-0229. Available from doi: 10.1016/j.gpb.2016.05.004.
3. LI, Heng; HOMER, Nils. A Survey of Sequence Alignment Algorithms for Next-Generation Sequencing. *Briefings in Bioinformatics*. 2010, vol. 11, no. 5, pp. 473–483. ISSN 1477-4054. Available from doi: 10.1093/bib/bbq015.
4. LI, Heng; HANDSAKER, Bob; WYSOKER, Alec; FENNELL, Tim; RUAN, Jue; HOMER, Nils; MARTH, Gabor; ABECASIS, Goncalo; DURBIN, Richard; 1000 GENOME PROJECT DATA PROCESSING SUBGROUP. The Sequence Alignment/Map Format and SAMtools. *Bioinformatics*. 2009, vol. 25, no. 16, pp. 2078–2079. ISSN 1367-4811. Available from doi: 10.1093/bioinformatics/btp352.
5. DUITAMA, Jorge; ZABLOTSKAYA, Alena; GEMAYEL, Rita; JANSEN, An; BELET, Stefanie; VERMEESCH, Joris R.; VERSTREPEN, Kevin J.; FROYEN, Guy. Large-Scale Analysis of Tandem Repeat Variability in the Human Genome. *Nucleic Acids Research* [online]. 2014, vol. 42, no. 9, pp. 5728–5741 [visited on 2026-05-20]. ISSN 0305-1048. Available from doi: 10.1093/nar/gku212.
6. LI, Heng. Minimap2: Pairwise Alignment for Nucleotide Sequences. *Bioinformatics* [online]. 2018, vol. 34, no. 18, pp. 3094–3100 [visited on 2026-03-30]. ISSN 1367-4803. Available from doi: 10.1093/bioinformatics/bty191.

7. *Maximum Read Length for Illumina Sequencing Platforms | Illumina Knowledge* [https://knowledge.illumina.com/instrumentation/general/instrumentation-general-reference_material-list/000002826]. 2026. [visited on 2026-03-30].
8. JAIN, Miten; KOREN, Sergey; MIGA, Karen H.; QUICK, Josh; RAND, Arthur C.; SASANI, Thomas A.; TYSON, John R.; BEGGS, Andrew D.; DILTHEY, Alexander T.; FIDDES, Ian T.; MALLA, Sunir; MARRIOTT, Hannah; NIETO, Tom; O'GRADY, Justin; OLSEN, Hugh E.; PEDERSEN, Brent S.; RHIE, Arang; RICHARDSON, Hollian; QUINLAN, Aaron R.; SNUTCH, Terrance P.; TEE, Louise; PATEN, Benedict; PHILLIPPY, Adam M.; SIMPSON, Jared T.; LOMAN, Nicholas J.; LOOSE, Matthew. Nanopore Sequencing and Assembly of a Human Genome with Ultra-Long Reads. *Nature Biotechnology* [online]. 2018, vol. 36, no. 4, pp. 338–345 [visited on 2026-03-30]. ISSN 1546-1696. Available from doi: 10.1038/nbt.4060.
9. DISHUCK, Philip C; ROZANSKI, Allison N; LOGSDON, Glennis A; PORUBSKY, David; EICHLER, Evan E. GAVISUNK: Genome Assembly Validation via Inter-SUNK Distances in Oxford Nanopore Reads. *Bioinformatics* [online]. 2023, vol. 39, no. 1, btac714 [visited on 2026-05-19]. ISSN 1367-4811. Available from doi: 10.1093/bioinformatics/btac714.
10. ALBERTS, Bruce; JOHNSON, Alexander; LEWIS, Julian; RAFF, Martin; ROBERTS, Keith; WALTER, Peter. *Molecular Biology of the Cell*. 4th. Garland Science, 2002. ISBN 978-0-8153-3218-3 978-0-8153-4072-0.
11. Ribosomal DNA and the Nucleolus in the Context of Genome Organization | Request PDF. *ResearchGate* [online]. [N.d.] [visited on 2026-05-20]. Available from doi: 10.1007/s10577-018-9600-5.
12. STULTS, Dawn M.; KILLEN, Michael W.; PIERCE, Heather H.; PIERCE, Andrew J. Genomic Architecture and Inheritance of Human Ribosomal RNA Gene Clusters. *Genome Research* [online]. 2008, vol. 18, no. 1, pp. 13–18 [visited on 2026-05-20]. ISSN 1088-9051. Available from doi: 10.1101/gr.6858507.

BIBLIOGRAPHY

13. GONZALEZ, Iris L.; SYLVESTER, James E. Complete Sequence of the 43-Kb Human Ribosomal DNA Repeat: Analysis of the Intergenic Spacer. *Genomics* [online]. 1995, vol. 27, no. 2, pp. 320–328 [visited on 2026-05-20]. ISSN 0888-7543. Available from DOI: 10.1006/geno.1995.1049.
14. GIBBONS, John G.; BRANCO, Alan T.; YU, Shoukai; LEMOS, Bernardo. Ribosomal DNA Copy Number Is Coupled with Gene Expression Variation and Mitochondrial Abundance in Humans. *Nature Communications* [online]. 2014, vol. 5, no. 1, p. 4850 [visited on 2026-05-20]. ISSN 2041-1723. Available from DOI: 10.1038/ncomms5850.
15. MOROZOVA, Olena; MARRA, Marco A. Applications of Next-Generation Sequencing Technologies in Functional Genomics. *Genomics*. 2008, vol. 92, no. 5, pp. 255–264. ISSN 1089-8646. Available from DOI: 10.1016/j.ygeno.2008.07.001.
16. HEJAZI, Sepehr; AHSAN, Afrin; KASHANI, Mohammad; REUEL, Nigel F. Amplified DNA Heterogeneity Assessment with Oxford Nanopore Sequencing Applied to Cell Free Expression Templates. *bioRxiv* [online]. 2024, p. 2024.06.02.597048 [visited on 2026-05-20]. ISSN 2692-8205. Available from DOI: 10.1101/2024.06.02.597048.
17. *Nanopore Sequencing Accuracy* [<https://nanoporetech.com/platform/accuracy>]. [N.d.]. [visited on 2026-05-20].
18. STOLER, Nicholas; NEKRUTENKO, Anton. Sequencing Error Profiles of Illumina Sequencing Instruments. *NAR Genomics and Bioinformatics* [online]. 2021, vol. 3, no. 1, lqab019 [visited on 2026-05-20]. ISSN 2631-9268. Available from DOI: 10.1093/nargab/lqab019.
19. SCHIRMER, Melanie; D'AMORE, Rosalinda; IJAZ, Umer Z.; HALL, Neil; QUINCE, Christopher. Illumina Error Profiles: Resolving Fine-Scale Variation in Metagenomic Sequencing Data. *BMC Bioinformatics* [online]. 2016, vol. 17, no. 1, p. 125 [visited on 2026-05-20]. ISSN 1471-2105. Available from DOI: 10.1186/s12859-016-0976-y.

20. DELAHAYE, Clara; NICOLAS, Jacques. Sequencing DNA with Nanopores: Troubles and Biases. *PLoS ONE* [online]. 2021, vol. 16, no. 10, e0257521 [visited on 2026-05-20]. ISSN 1932-6203. Available from DOI: 10.1371/journal.pone.0257521.
21. AHMED, Nauman; BERTELS, Koen; AL-ARS, Zaid. A Comparison of Seed-and-Extend Techniques in Modern DNA Read Alignment Algorithms. In: *2016 IEEE International Conference on Bioinformatics and Biomedicine (BIBM)* [online]. 2016, pp. 1421–1428 [visited on 2026-05-20]. Available from DOI: 10.1109/BIBM.2016.7822731.
22. KWAN, James Z.J.; NGUYEN, Thomas F.; TEVES, Sheila S. TBP Facilitates RNA Polymerase I Transcription Following Mitosis. *RNA Biology* [online]. [N.d.], vol. 21, no. 1, pp. 42–51 [visited on 2026-05-20]. ISSN 1547-6286. Available from DOI: 10.1080/15476286.2024.2375097.
23. MOECKEL, Camille; MAREBOINA, Manvita; KONNARIS, Maxwell A.; CHAN, Candace S.Y.; MOURATIDIS, Ioannis; MONTGOMERY, Austin; CHANTZI, Nikol; PAVLOPOULOS, Georgios A.; GEORGAKOPOULOS-SOARES, Ilias. A Survey of K-Mer Methods and Applications in Bioinformatics. *Computational and Structural Biotechnology Journal* [online]. 2024, vol. 23, pp. 2289–2303 [visited on 2026-05-20]. ISSN 2001-0370. Available from DOI: 10.1016/j.csbj.2024.05.025.
24. CECHOVA, Monika; POTAPOVA, Tamara A.; RECHTSTEINER, Andreas; HICKEY, Glenn; MARI, Rebecca Serra; MASTORAS, Mira; MENENDEZ, Julian; POLÁKOVÁ, Nikol; HEBBAR, Prajna; RYABOV, Fedor; LOUCKS, Hailey; GROOT, Aljona; PAVLÍK, Tomáš; ASRI, Mobin; DONG, Shihua; YAN, Stephanie M.; LUCAS, Julian K.; SOLAR, Steven J.; BORCHERS, Matthew; MATTINGLY, Mark; MCKINNEY, Sean; KRÁTKÁ, Marie; MIKHAILOVA, Catherine; HANÁK, Ondřej; SAHA, Sohinee Tiffany; XU, Emily; ANTIPOV, Dmitry; KOREN, Sergey; EIZENGA, Jordan M.; MCNULTY, Brandy; GARDNER, Joshua M. V.; HILLAKER, Todd; VIOLICH, Ivo; MARKOVIC, Christopher; KRUGLYAK, Semyon; LEVY, Shawn; WOLF, Trevor; MITCHELL, Matthew W.; SCHEINFELDT, Laura; CHENG,

- Haoyu; ALEXANDROV, Ivan A.; MCCOY, Rajiv C.; PATEN, Benedict; PHILLIPPY, Adam M.; ZOOK, Justin M.; GERTON, Jennifer L.; FULTON, Robert S.; STITZIEL, Nathan O.; WANG, Ting; MARSCHALL, Tobias; GREIDER, Carol W.; MIGA, Karen H. Complete Genomes of a Multi-Generational Pedigree to Expand Studies of Genetic and Epigenetic Inheritance. *bioRxiv: The Preprint Server for Biology*. 2025, p. 2025.12.14.693655. ISSN 2692-8205. Available from DOI: 10.64898/2025.12.14.693655.
25. WICK, Ryan R. Badread: Simulation of Error-Prone Long Reads. *Journal of Open Source Software* [online]. 2019, vol. 4, no. 36, p. 1316 [visited on 2026-05-14]. ISSN 2475-9066. Available from DOI: 10.21105/joss.01316.
26. YANG, Chen; CHU, Justin; WARREN, René L; BIROL, Inanç. NanoSim: Nanopore Sequence Read Simulator Based on Statistical Characterization. *GigaScience* [online]. 2017, vol. 6, no. 4, gix010 [visited on 2026-05-14]. ISSN 2047-217X. Available from DOI: 10.1093/gigascience/gix010.
27. THORVALDSDÓTTIR, Helga; ROBINSON, James T.; MESIROV, Jill P. Integrative Genomics Viewer (IGV): High-Performance Genomics Data Visualization and Exploration. *Briefings in Bioinformatics* [online]. 2013, vol. 14, no. 2, pp. 178–192 [visited on 2026-05-20]. ISSN 1467-5463. Available from DOI: 10.1093/bib/bbs017.